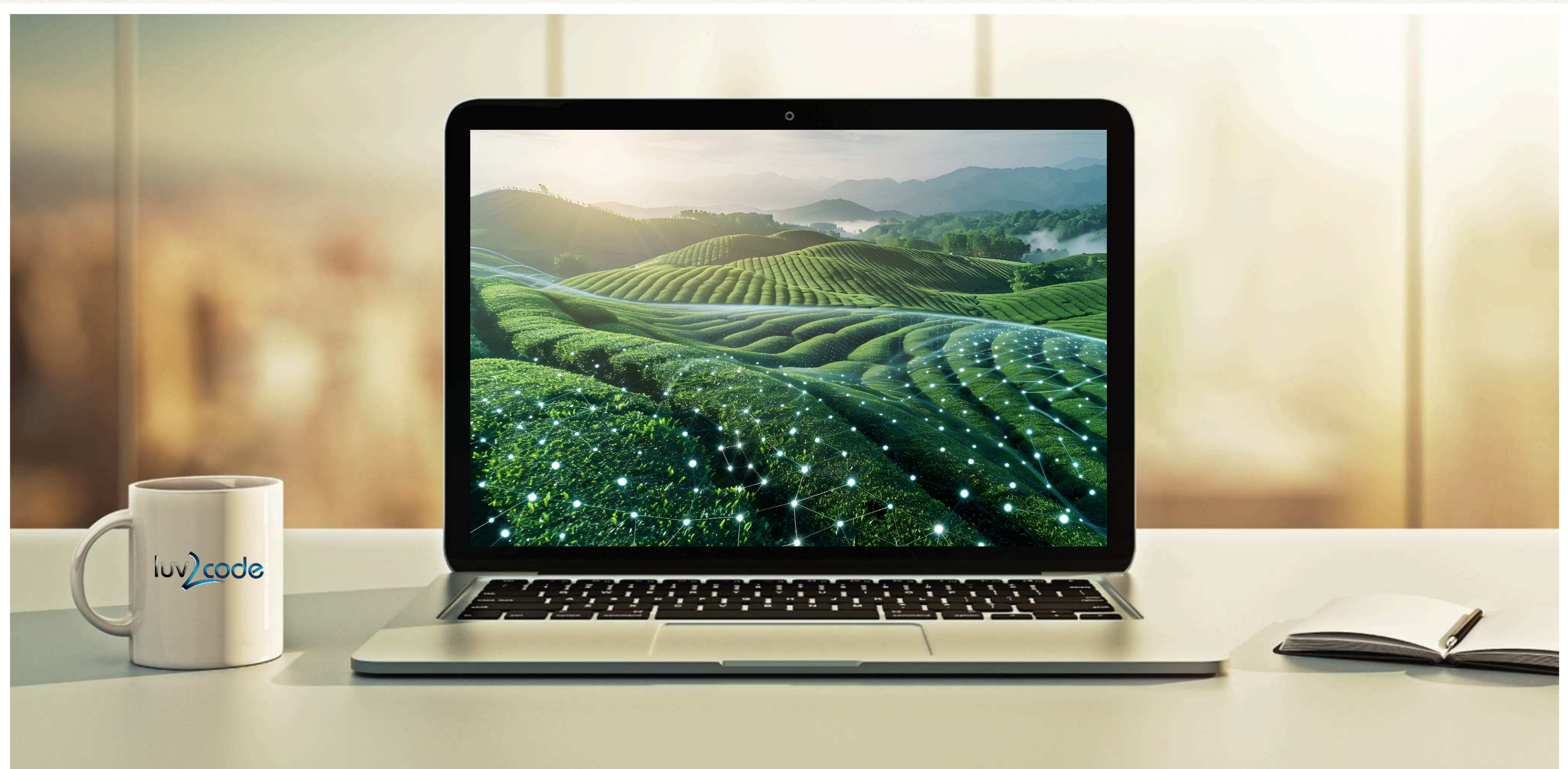


Project 4

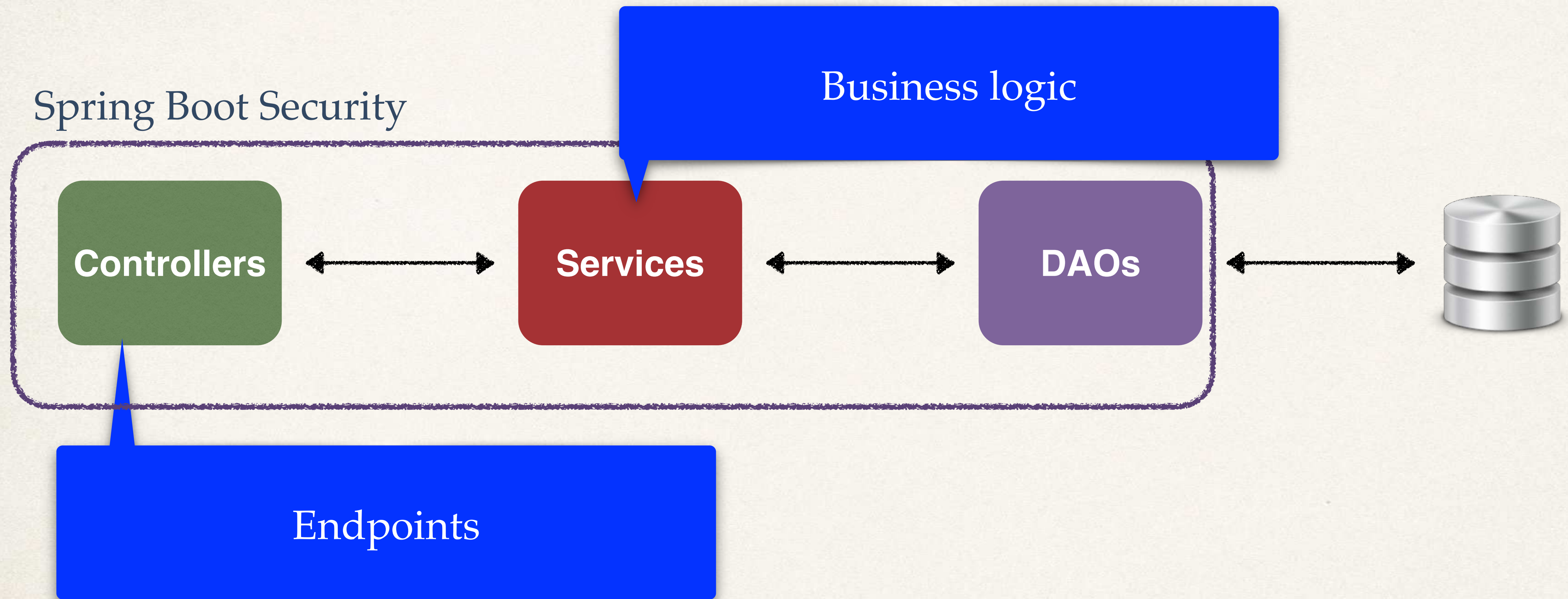


Project 4

- Project four we will be switching our focus to a deep Employee / Admin Todo project.
- New Information will include:
 - Professional (MySQL) Databases
 - Authentication / Authorization using JWTs
 - Admin roles for global permissions
 - User registration.

**Continued learning from
other projects**

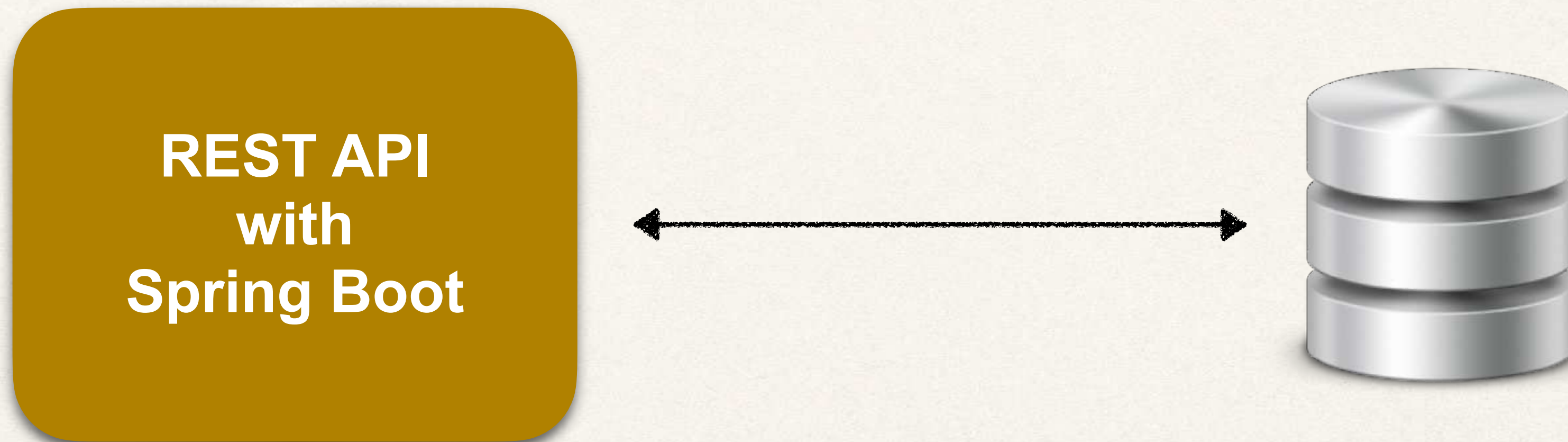
Application Architecture



Project Overview

Real-Time Project

REST API with Spring Boot
that connects to a professional database



Requirements

From the Boss

Create multiple REST APIs for the Todo Directory:

1. Users who can request their own information.
2. Todos that can be fetched, created, updated and deleted by their user.
3. Admins who can get all users, promote users to admin and delete non-admin users.
4. Authentication for registration & login using JWTs.

All using a MySQL database

Step 1: API Requirements

step-by-step

Create entities & MySQL database.

Create the necessary requirements to get started:

- Create Todo Entity in code.
- Create User Entity in code.
- Create Authority Entity in code.
- Add the mappings between each entity.
- Setup MySQL.

Step 2: API Requirements

step-by-step

Create REST APIs for the Authentication & Authorization

- Install JWT dependencies.
- Setup JWTs within our project.
- Create and add security configurations to project
- Add Authentication Service functionality.
- Add Authentication Controller functionality.

Step 3: API Requirements

step-by-step

Create REST APIs for the Users

- Add User Request & Response DTOs
- Add User Service functionality.
- Add User Controller functionality.

Step 4: API Requirements

step-by-step

Create REST APIs for the Todos

- Add Todo Request & Response DTOs
- Add Todo Service functionality.
- Add Todo Controller functionality.

Step 5: API Requirements

step-by-step

Create REST APIs for the Admin roles.

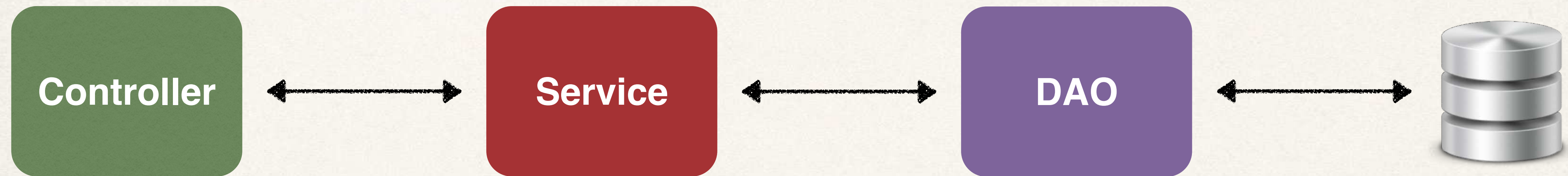
- Add Admin roles to security configurations.
- Add Admin Service functionality.
- Add Admin Controller functionality.

Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer .
2. Create entities.
3. Implement creating & reading of JWTs.
4. Add authentication & authorization to product.
5. Add users to product.
6. Add Todo to product.
7. Add admin roles to product.

Application Architecture



Create Project - Spring Initizer



Development Process

Step-By-Step

1. Create Spring Boot project using Spring Initializer .
2. Create entities.
3. Implement creating & reading of JWTs.
4. Add authentication & authorization to product.
5. Add users to product.
6. Add Todo to product.
7. Add admin roles to product.

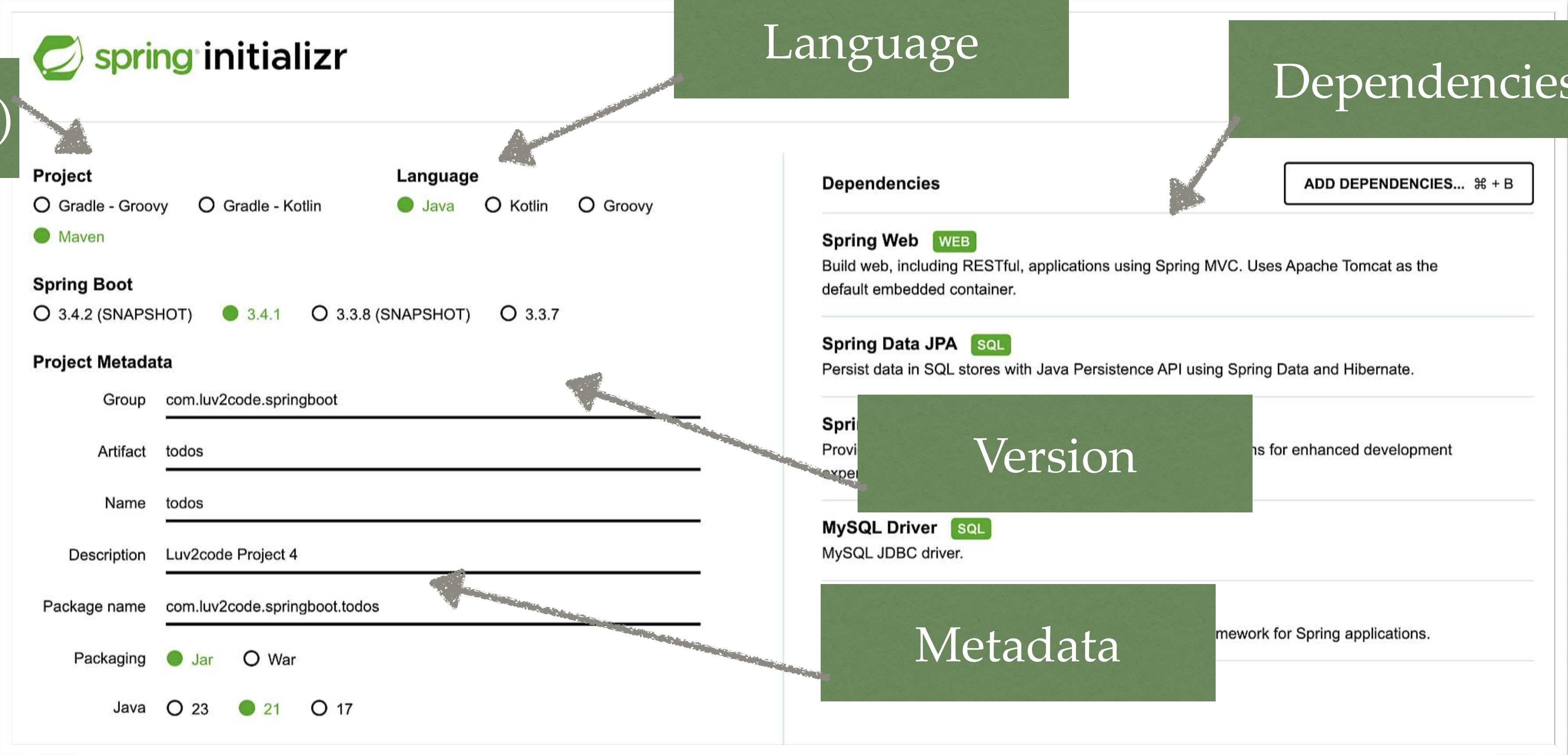
Spring Initializr

- Here we can set the project configurations for our upcoming Spring project.
- Allows us to choose the project, programming language, Spring Boot version, dependencies and metadata for our application.

start.spring.io

Spring Initializr

- Setup our project using Spring Initializr



The image shows the Spring Initializr web form with several green callout boxes and arrows pointing to specific fields:

- Project (maven)**: Points to the **Project** section where **Maven** is selected.
- Language**: Points to the **Language** section where **Java** is selected.
- Dependencies**: Points to the **ADD DEPENDENCIES...** button.
- Version**: Points to the **3.4.1** version selection in the **Spring Boot** section.
- Metadata**: Points to the **Package name** field.

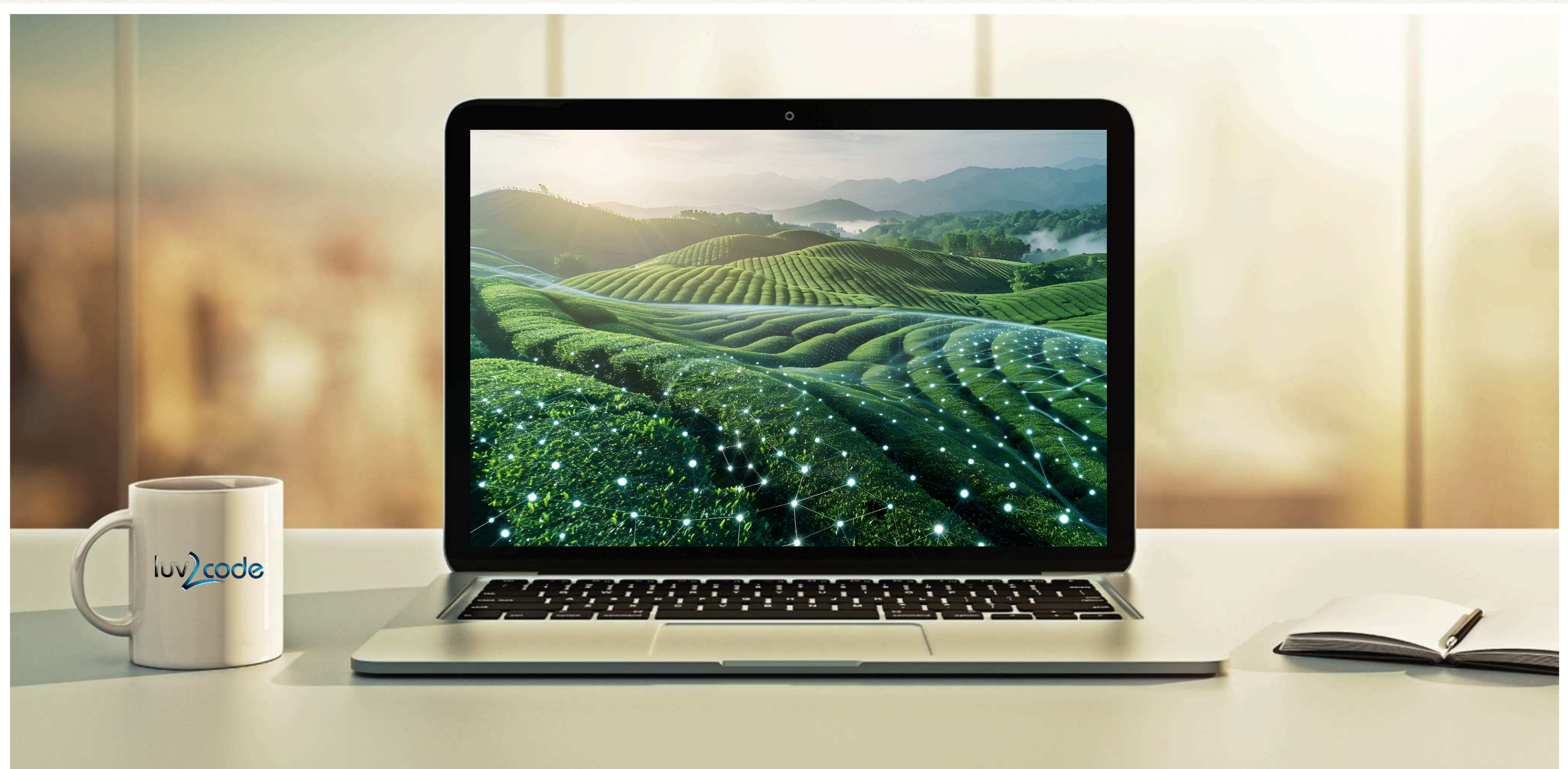
The form fields are as follows:

- Project**: ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ **Maven**
- Language**: ☒ **Java**, ☐ Kotlin, ☐ Groovy
- Spring Boot**: ☐ 3.4.2 (SNAPSHOT), ☒ **3.4.1**, ☐ 3.3.8 (SNAPSHOT), ☐ 3.3.7
- Project Metadata**:
 - Group:
 - Artifact:
 - Name:
 - Description:
 - Package name:
 - Packaging: ☒ **Jar**, ☐ War
 - Java: ☐ 23, ☒ **21**, ☐ 17
- Dependencies**:
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
 - Spring** (WEB): Provides support for enhanced development.
 - MySQL Driver** (SQL): MySQL JDBC driver.
 - Spring Framework** (WEB): Framework for Spring applications.

Spring Initializr

- Spring Initializr will download a .zip file onto our machine.
- We will need to unzip the .zip file.
- Open the project on IntelliJ.
- Download all dependencies using Maven.
- Sounds fun - Let's go ahead and dive in!

Create Entities



What entities do we need?

- Todo Entity
- User Entity
- Authority Entity



Todo Entity

File: Todo.java

```
@Table(name = "todos")
@Entity
public class Todo {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(nullable = false)
    private long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String description;

    @Column(nullable = false)
    private int priority;

    @Column(nullable = false)
    private boolean complete;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "owner_id", nullable = false)
    private User owner;
}
```

...

File: Todo.java (continued)

```
// Constructors
public Todo() {}

public Todo(String title, String description,
            int priority, boolean complete, User owner) {

    this.title = title;
    this.description = description;
    this.priority = priority;
    this.complete = complete;
    this.owner = owner;
}

<<getters & setters for each item>>
}
```

todos	
id	bigint
title	varchar NN
description	varchar NN
priority	int NN
complete	boolean NN
owner_id	bigint NN

User Entity

File: User.java

```
@Table(name = "users")
@Entity
public class User implements UserDetails {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(nullable = false)
    private long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @Column(unique = true, length = 100, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    ...
}
```

File: User.java (continued)

```
@CreationTimestamp
@Column(updatable = false, name = "created_at")
private Date createdAt;

@UpdateTimestamp
@Column(name = "updated_at")
private Date updatedAt;

@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "user_authorities",
    joinColumns = @JoinColumn(name = "user_id"))
private List<Authority> authorities;

@OneToMany(mappedBy = "owner",
    cascade = CascadeType.ALL, orphanRemoval = true)
private List<Todo> todos;

public User() {}
}
```

users	
id	bigint
first_name	varchar NN
last_name	varchar NN
email	varchar(100) NN
password	varchar NN
created_at	timestamp NN
updated_at	timestamp

Authority Entity

File: Authority.java

```
@Embeddable
public class Authority implements GrantedAuthority {

    private String authority;

    public Authority() {}

    public Authority(String authority) {

        this.authority = authority;
    }

    @Override
    public String getAuthority() {
        return authority;
    }
}
```

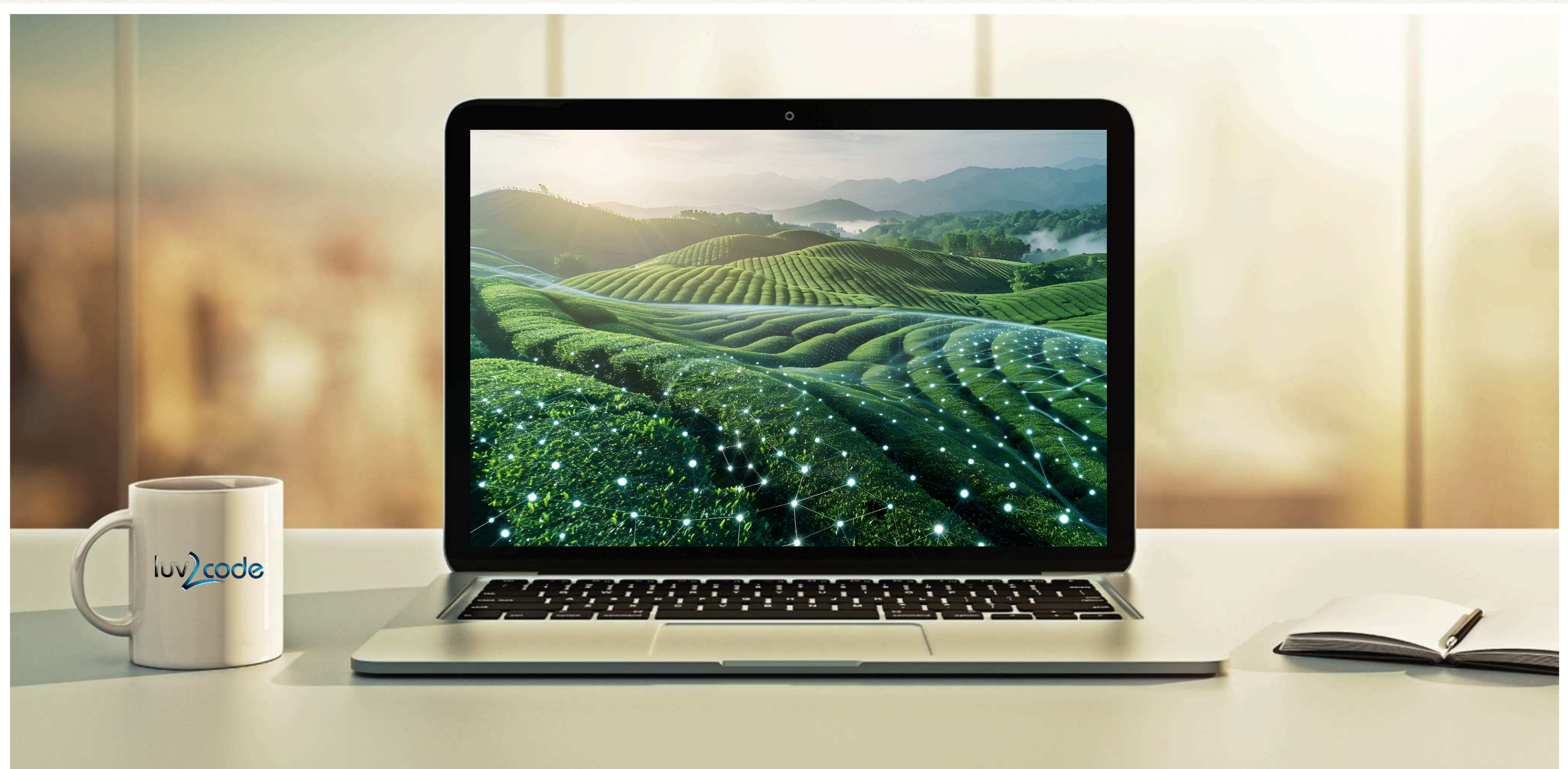
- @Embeddable indicates this class can be embedded as a component inside other entities.
- Implements Granted Authority which is a core interface in Spring Security, representing permissions granted to an authentication object.

File: User.java

```
@ElementCollection(fetch = FetchType.EAGER)
@CollectionTable(name = "user_authorities",
    joinColumns = @JoinColumn(name = "user_id"))
private List<Authority> authorities;
```

user_authorities	
user_id	bigint
authority	varchar NN

Setup MySQL Database



MySQL Setup Options:

- 1. You can download MySQL onto your machine and install from there.
- 2. Use Docker to host a container running MySQL for us.
- The second option is a really fast implementation and we can delete/create the MySQL database as often as we like.
- So, we will be proceeding with a Docker setup.

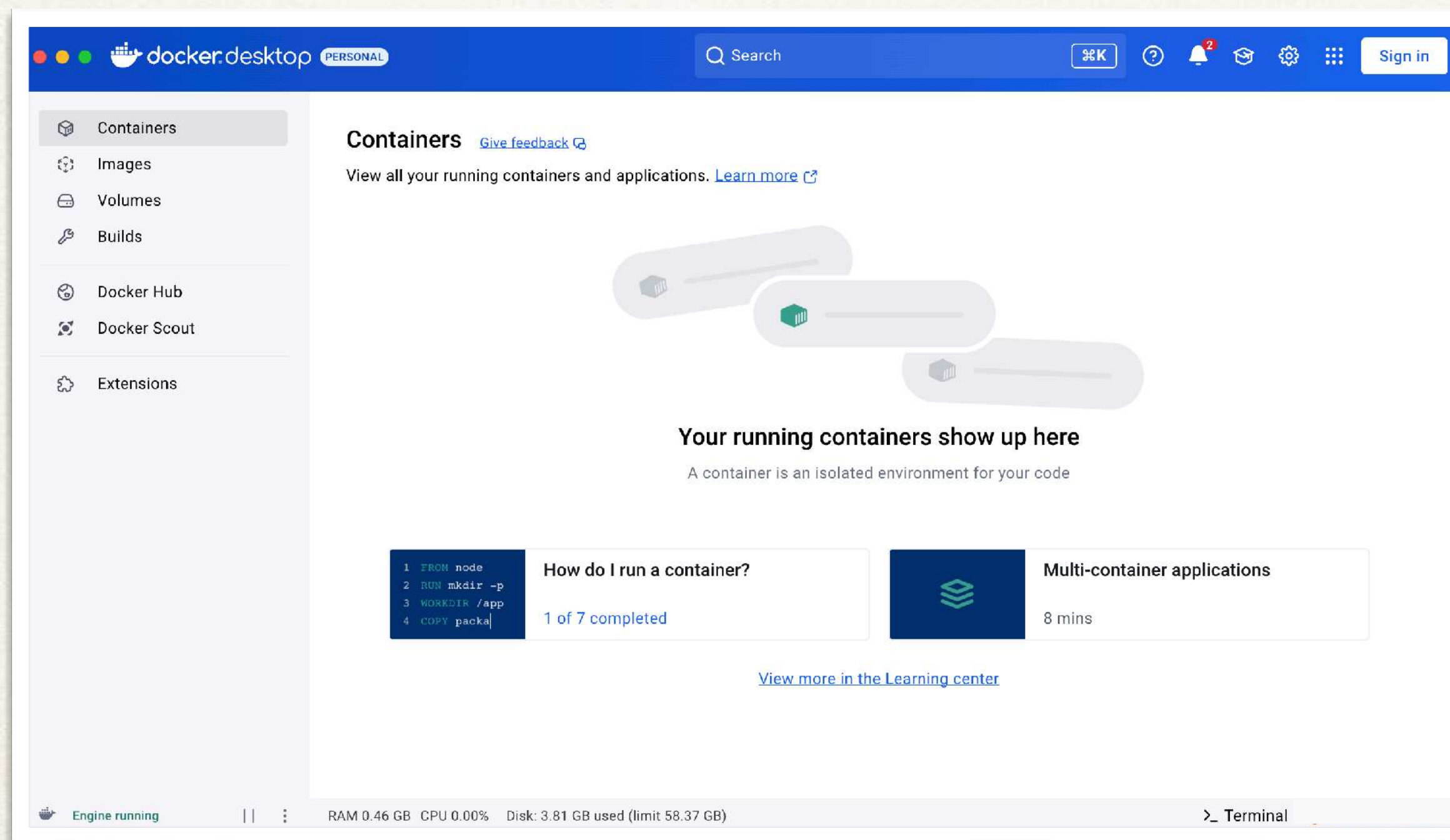
What is Docker?

- Docker is a containerization platform that allows you to package applications into something called containers.
- These containers ensure that our applications run consistently across environments.
- We will be only working with two commands:
 - Containers: Isolated environments (where MySQL will run locally).
 - Images: Blueprints for the containers (how to create MySQL)

Easiest way to get Docker

- Go to docker.com
- Download Docker Desktop. ~2GB download.
- Compatible with Windows, Mac and Linux.

Download Docker Desktop



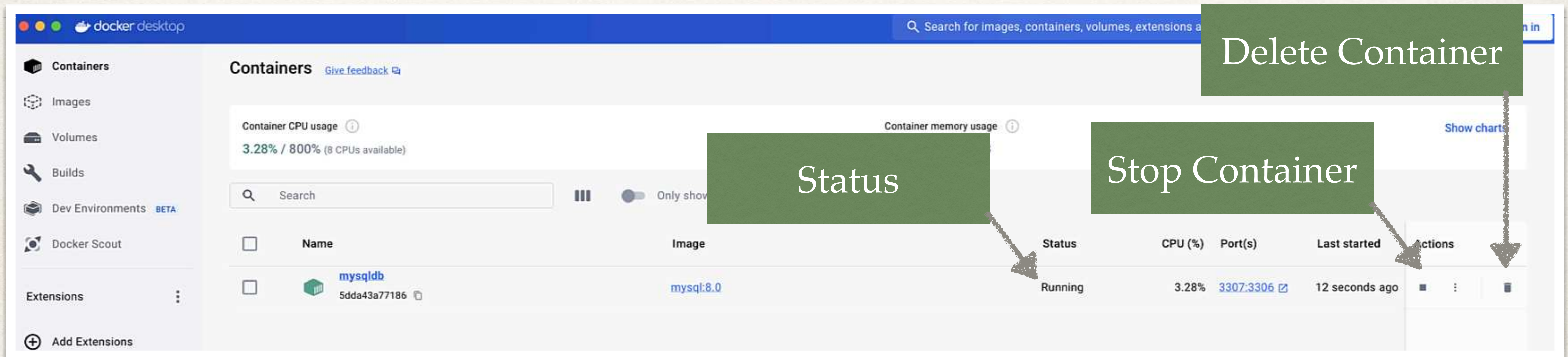
Once Docker is installed it is one command to get MySQL running

MySQL Setup Options:

- Command to spin up a MySQL Docker command:

```
docker run -d -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=tododb --name mysqlldb  
-p 3307:3306 mysql:8.0
```

- MySQL now running within Docker Desktop



Add MySQL Configurations

- We need to add properties to connect to MySQL database:

File: application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3307/tododb?serverTimezone=UTC&allowPublicKeyRetrieval=true&useSSL=false
spring.datasource.username=root
spring.datasource.password=secret

spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
```

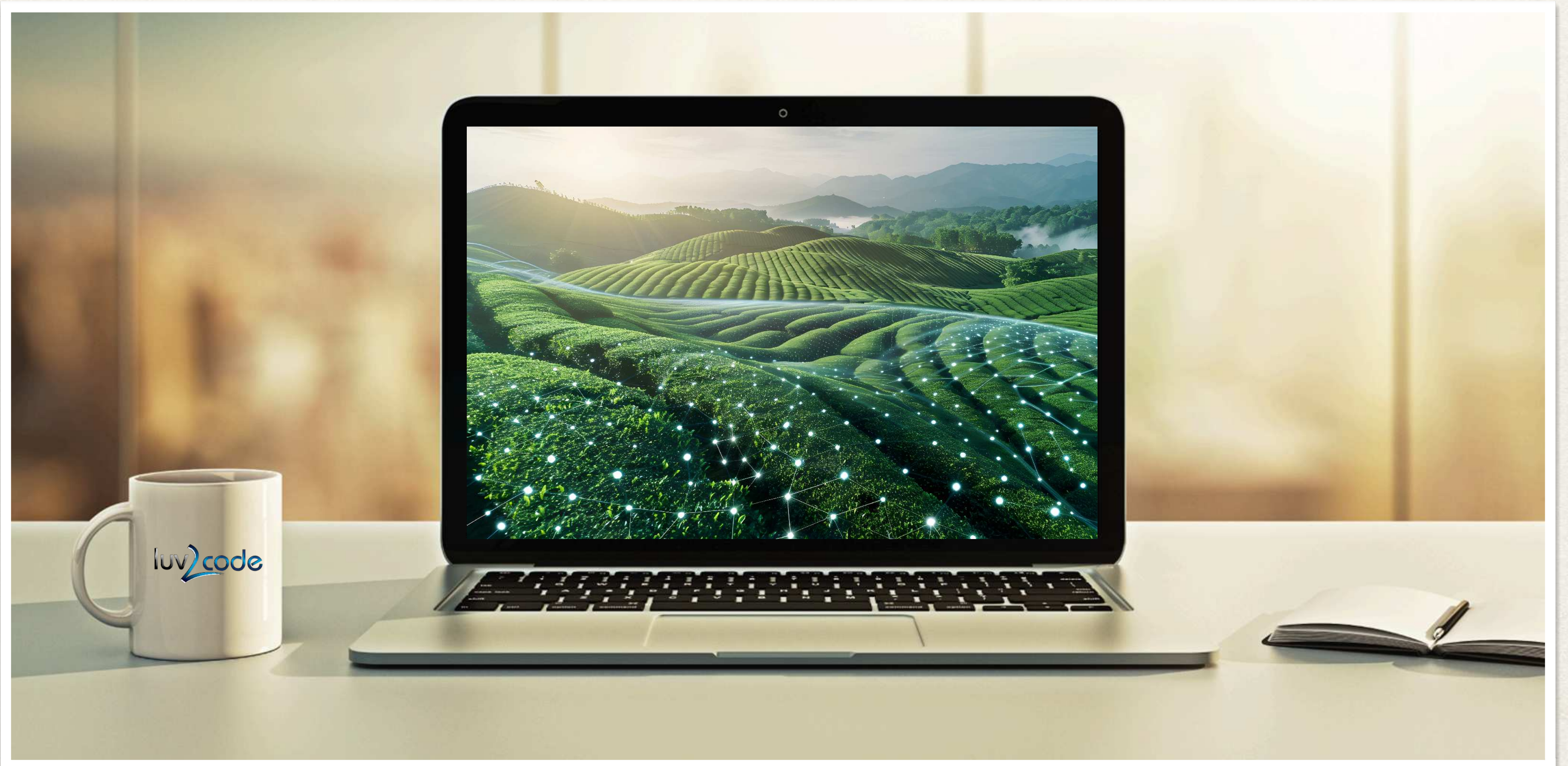
- spring.datasource.url = configures MySQL database connection to our local database instance
- spring.datasource.username = sets our database username
- spring.datasource.password = sets our database password
- spring.jpa.hibernate.ddl.auto = controls how the database schema is handled. Update means we will automatically update the table based on entities but won't drop any tables.
- spring.jpa.open-in-view = closes JPA EntityManager when transactions end.

DBeaver

- GUI to see our MySQL data easily.
- Supports many other databases as well.
- Community edition is FREE and open source.
- Over 40k+ stars on GitHub



Swagger Implementation



Install OpenAPI for Swagger

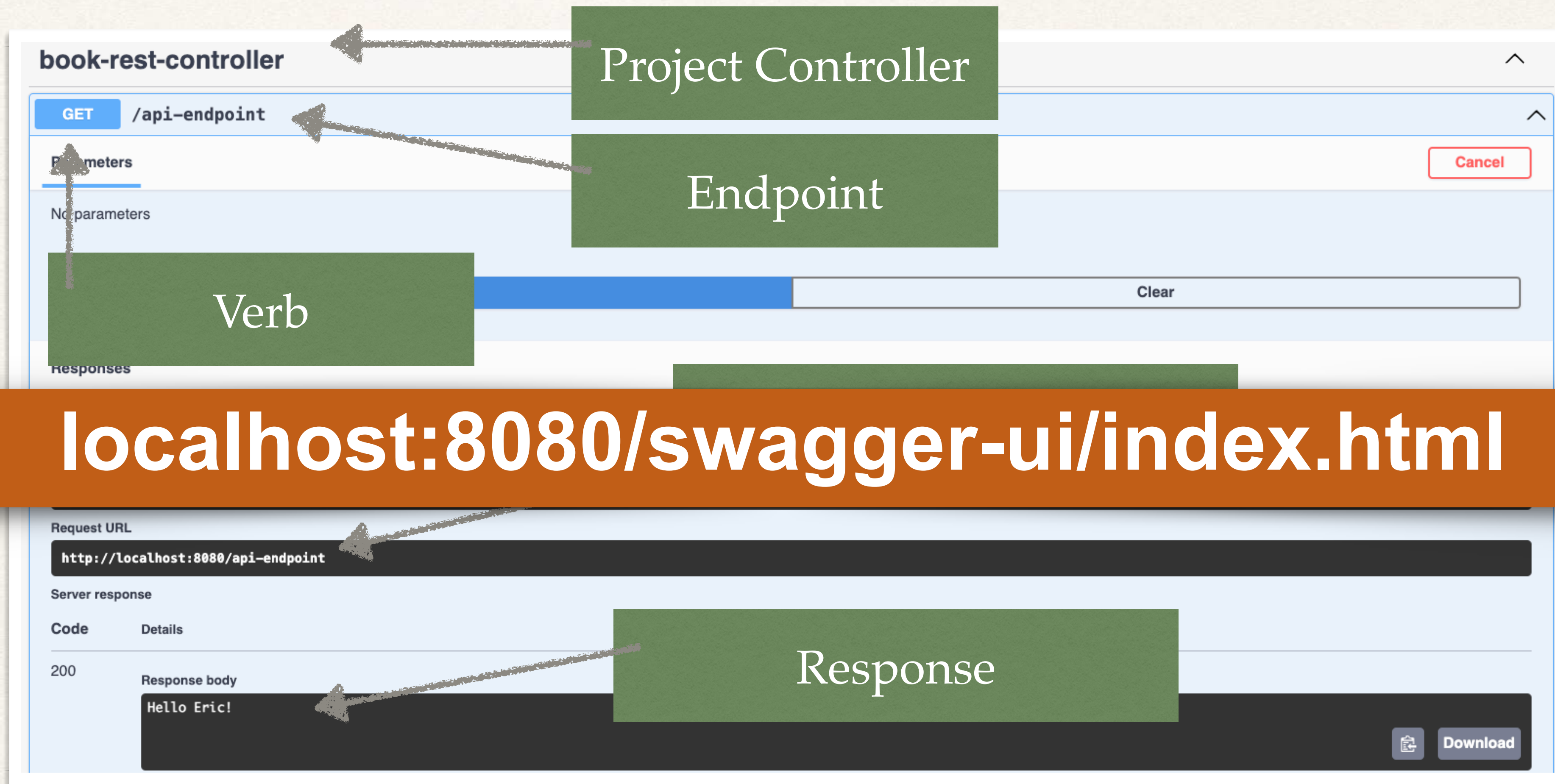
```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>x.y.z</version>  
</dependency>
```

We will provide
version number in upcoming video

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```


What is Swagger?

- Instead of going straight to the API endpoint, we can go to Swagger's.



What is Swagger?

- Path is hard to remember:

- localhost:8080/swagger-ui/index.html

Too long to remember

- We can fix this to redirect based off of /docs.

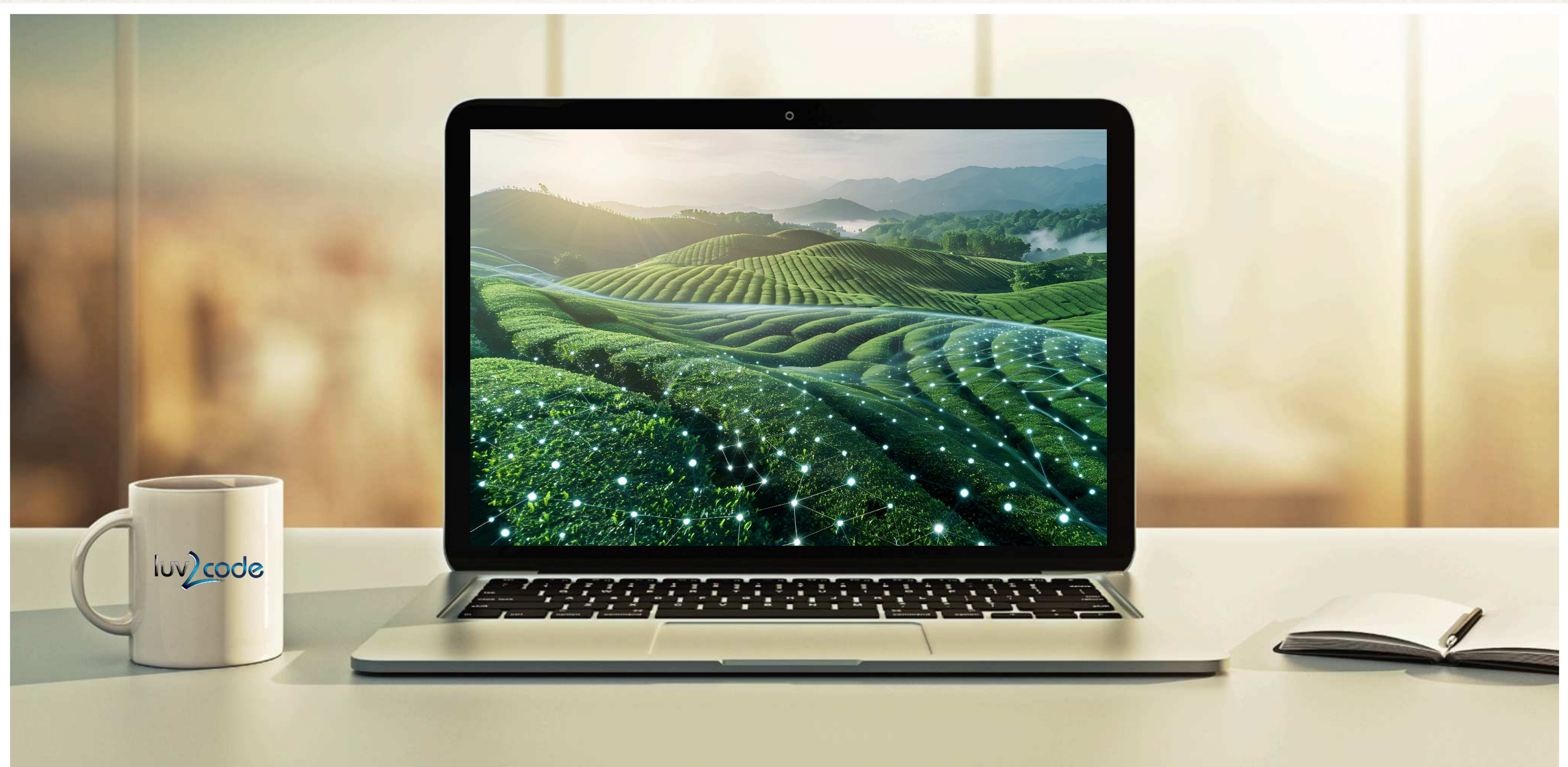
- New endpoint for Swagger: localhost:8080/docs

Much better :-)

File: application.properties

```
springdoc.swagger-ui.path=/docs
```


What are JWTs?



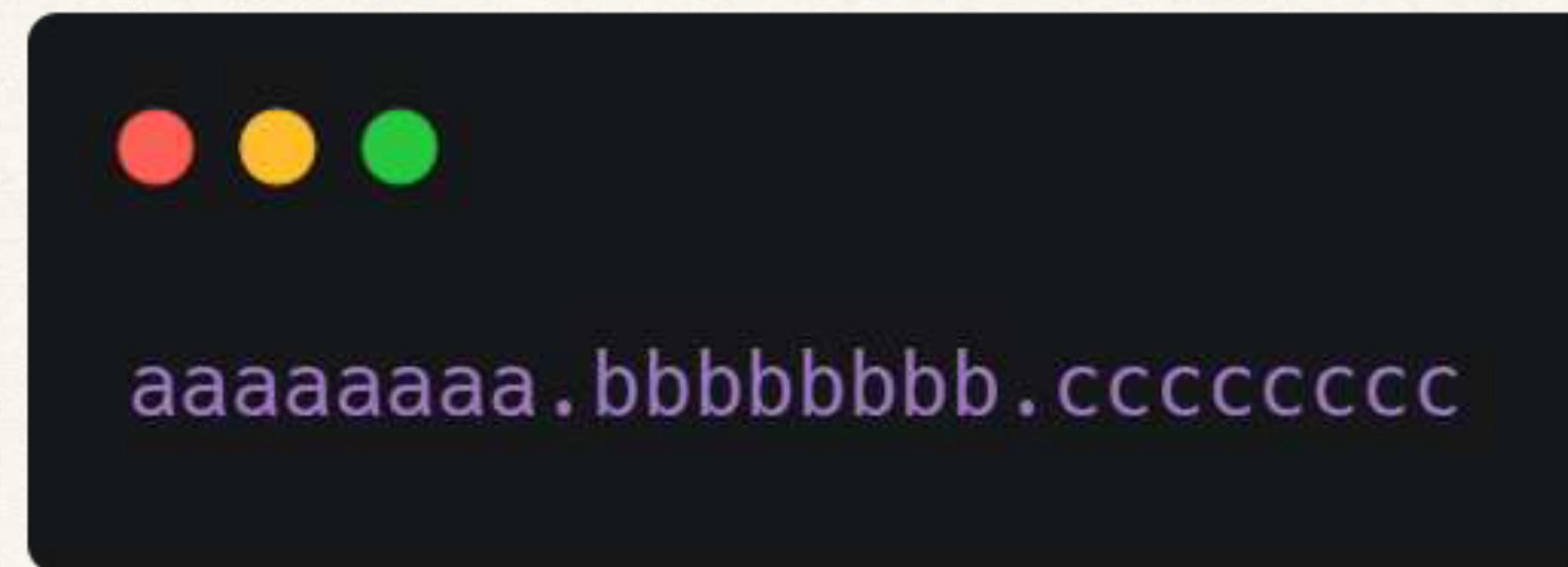
What is a JSON Web Token?

- JSON Web Token is a self-contained way to securely transmit data and information between two parties using a JSON Object.
- JSON Web Tokens can be trusted because each JWT can be digitally signed, which in return allows the server to know if the JWT has been changed at all
- JWT should be used when dealing with authorization
- JWT is a great way for information to be exchanged between the server and client

What is a JSON Web Token?

- A JSON Web Token is created of three separate parts separated by dots (.) which include:
 - Header (a)
 - Payload (b)
 - Signature (c)

JWT Structure



JWT Header

- A JWT header usually consist of two parts:
 - (alg) The algorithm for signing
 - (typ) The specific type of token
- The JWT header is then encoded using Base64 to create the first part of the JWT (a)

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

JWT Structure

```
aaaaaaaa.bbbbbbbb.cccccccc
```


JWT Payload

- A JWT Payload consists of the data. The Payloads data contains claims, data about the request or user
- The JWT Payload is then encoded using Base64 to create the second part of the JWT (b)

JWT Structure



aaaaaaaa.bbbbbbbb.cccccccc

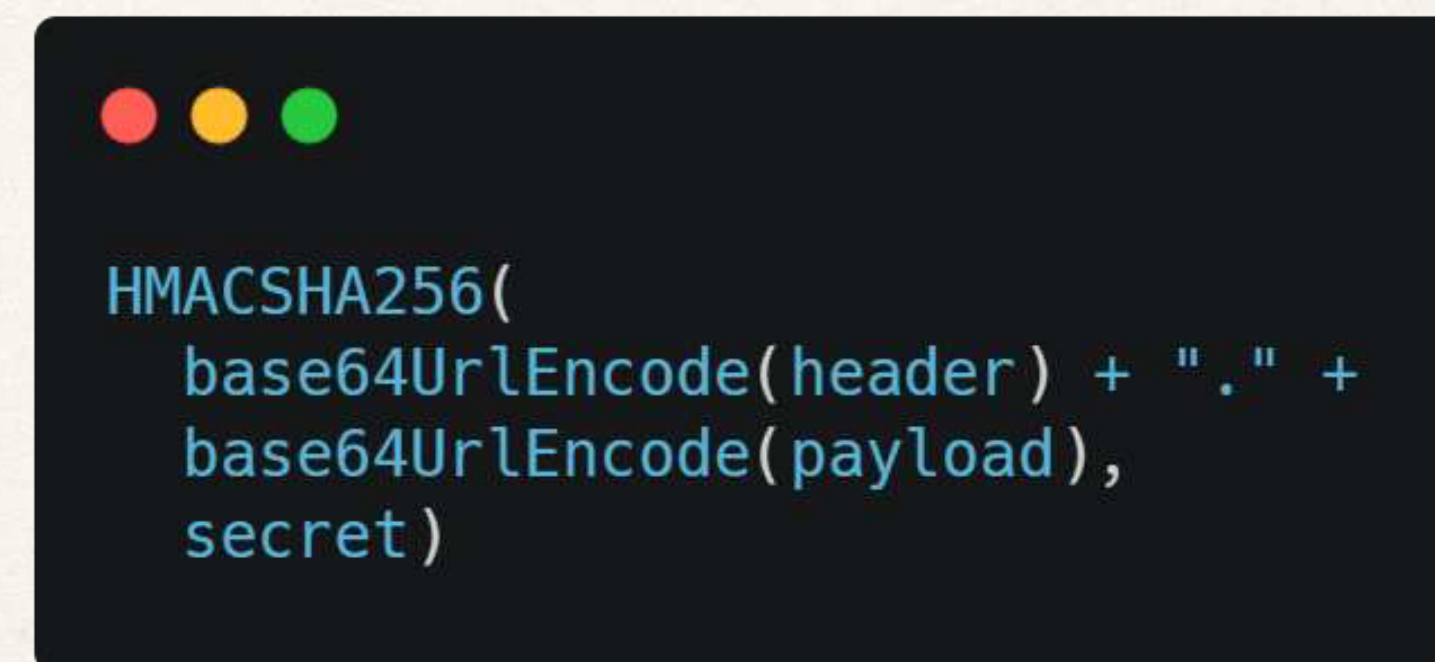
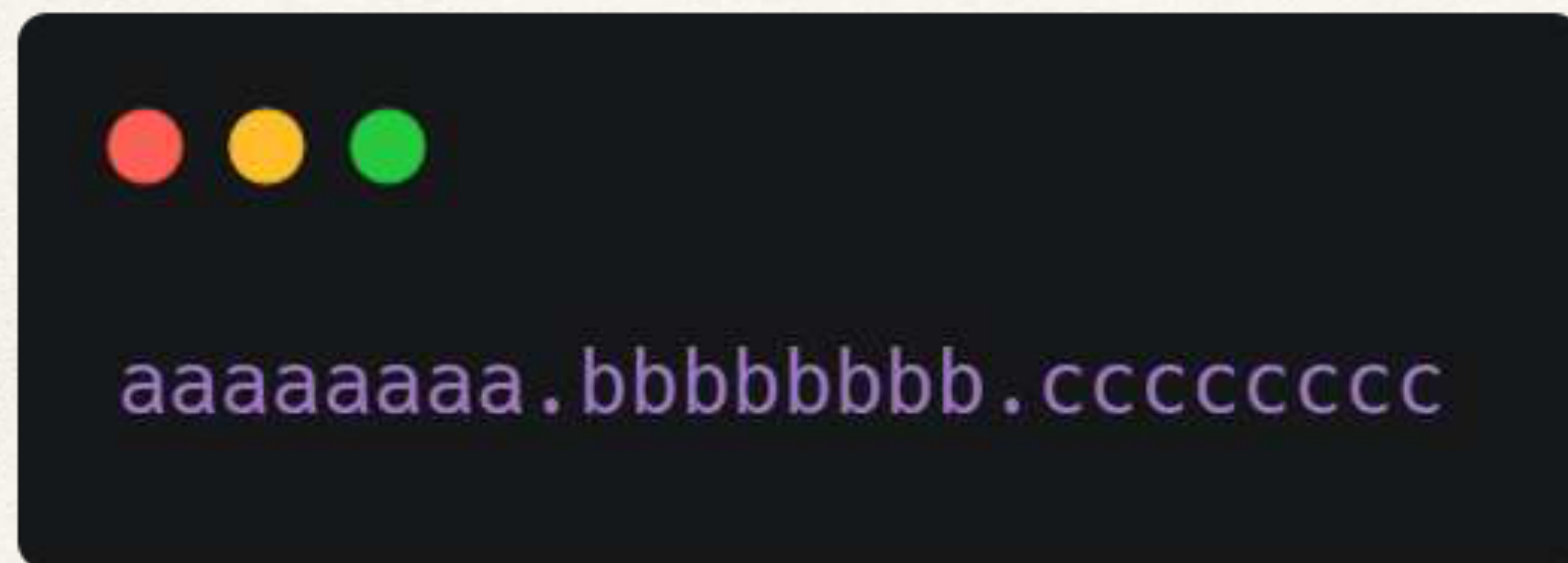


```
{  
  "sub": "1234567890",  
  "name": "Eric Roby",  
  "given_name": "Eric",  
  "family_name": "Roby",  
  "email": "codingwithroby@gmail.com",  
  "admin": true  
}
```

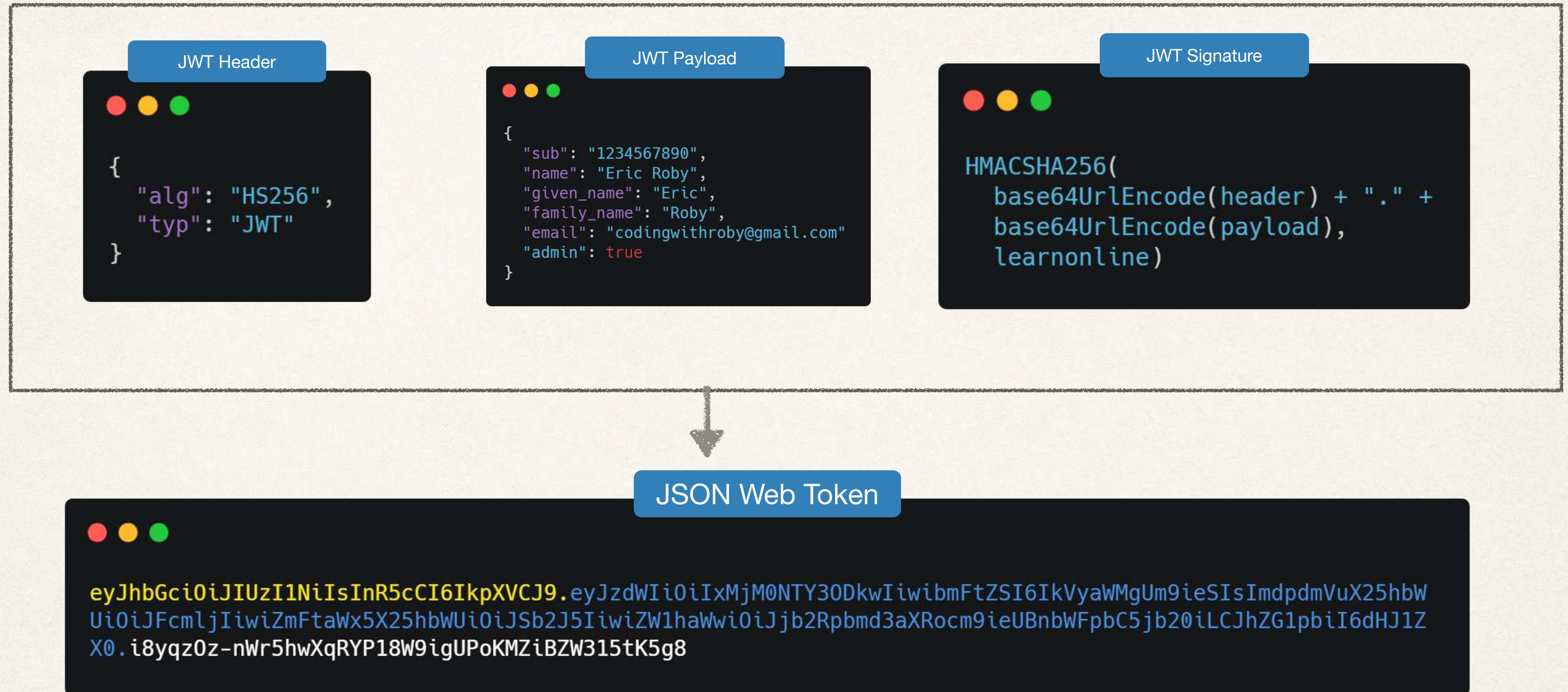

JWT Signature

- A JWT Signature is created by using the algorithm in the header to hash out the encoded header, encoded payload with a secret.
- The secret can be anything, but is saved somewhere on the server that the client does not have access to
- The signature is the third and final part of a JWT (c)

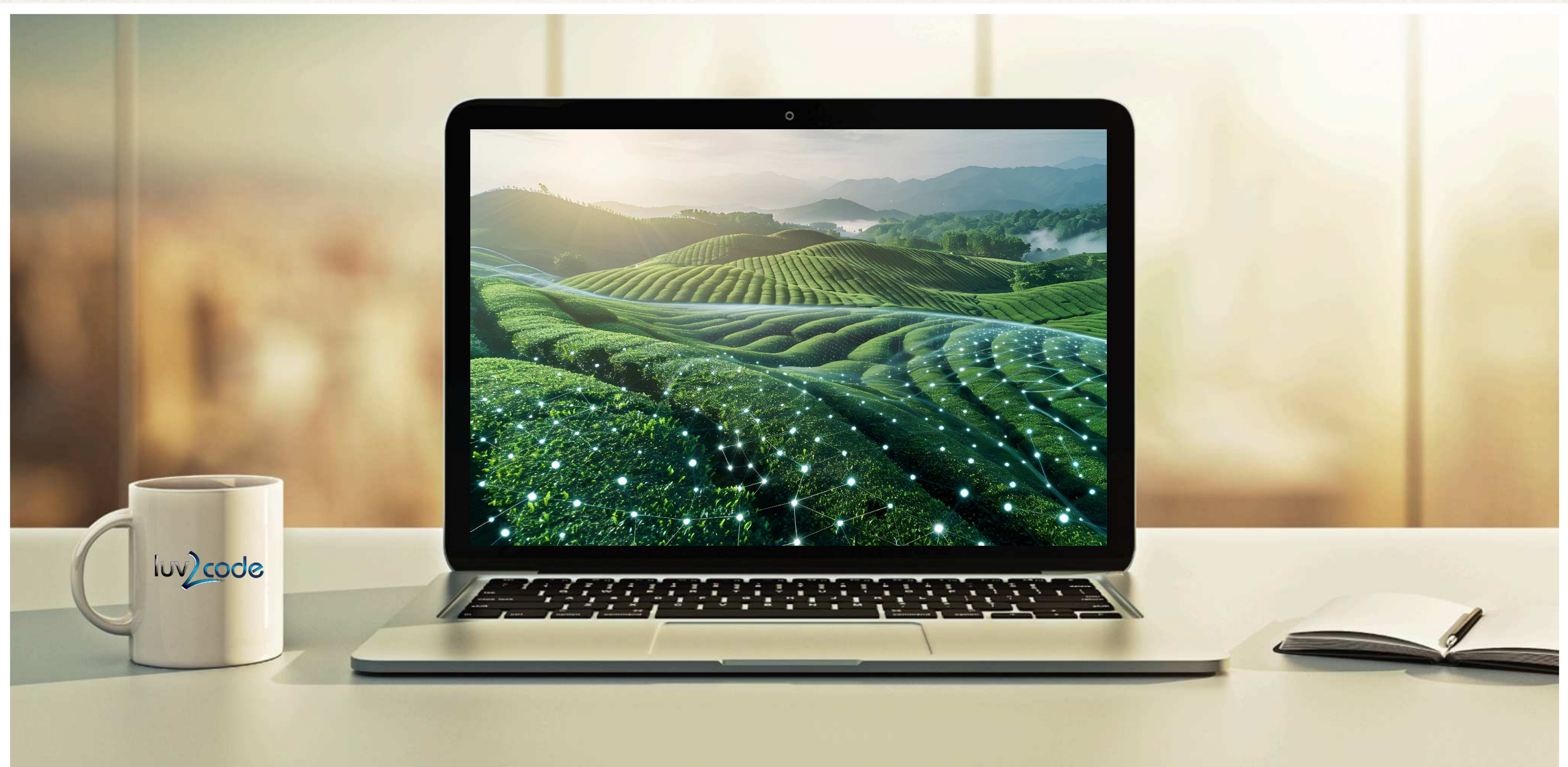
JWT Structure



JWT Example



Setup JWTs



Development Process

Step-By-Step

1. Install JWT into our Java Spring Boot Project

2. Create JWT Service.

1. Token Parsing

2. Token Validation

3. Token Generation

3. JWT Security Authentication Filter

1. Intercept every HTTP request.

2. If endpoint require authentication, validate token.

Install JJWT for JSON Web Tokens

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>x.y.z</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>x.y.z</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>x.y.z</version>
  <scope>runtime</scope>
</dependency>
```

We will provide
version numbers in upcoming video

JJWT Configurations

- We need to add properties for our JWTs

File: application.properties

```
spring.jwt.secret=370fc2ad32927ff329d2807b020a92cbde3d349b44c19304d3c8e8b107365042  
spring.jwt.expiration=900000
```

- spring.jwt.secret = Secret to sign and verify JWTs. Used for integrity and authenticity of the tokens.
- spring.jwt.expiration = Expiration of the token after creation in ms.
900,000ms = 15 minutes.

Development Process

Step-By-Step

1. Install JWT into our Java Spring Boot Project

2. Create JWT Service.

1. Token Parsing

2. Token Validation

3. Token Generation

3. JWT Security Authentication Filter

1. Intercept every HTTP request.

2. If endpoint require authentication, validate token.

We need to create a JWT Service

- Token Generation
 - Creates JWTs for authenticated users.
 - Signs token using HS256 algorithm
- Token Validation
 - Verifies if a token is valid or expired.
 - Check if token belongs to correct user.
- Token Parsing
 - Extracts username, claims and expiration date.

JWT Interface

File: JwtService.java

```
public interface JwtService {  
    String extractUsername(String token);  
    Boolean isValidToken(String token, UserDetails userDetails);  
    String generateToken(Map<String, Object> claims, UserDetails userDetails);  
}
```

2. Create JWT Service.

1. Token Parsing
2. Token Validation
3. Token Generation

JWT Service Impl

File: JwtServiceImpl.java

```
@Service
public class JwtServiceImpl implements JwtService{

    @Value("${spring.jwt.secret}")
    private String SECRET_KEY;

    @Value("${spring.jwt.expiration}")
    private long JWT_EXPIRATION;

    @Override
    public String generateToken(Map<String, Object> claims, UserDetails userDetails) {
        return Jwts.builder()
            .claims(claims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + JWT_EXPIRATION))
            .signWith(getSigningKey(), Jwts.SIG.HS256)
            .compact();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(SECRET_KEY);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

2. Create JWT Service.

1. Token Parsing

2. Token Validation

3. Token Generation ✓

JWT Service Impl

File: JwtServiceImpl.java

```
@Service
public class JwtServiceImpl implements JwtService{

    ...

    @Override
    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    private <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }
}
```

2. Create JWT Service.

1. Token Parsing ✓

2. Token Validation

3. Token Generation ✓

JWT Service Impl

File: JwtServiceImpl.java

```
@Service
public class JwtServiceImpl implements JwtService{

    ...

    @Override
    public boolean isValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
    }

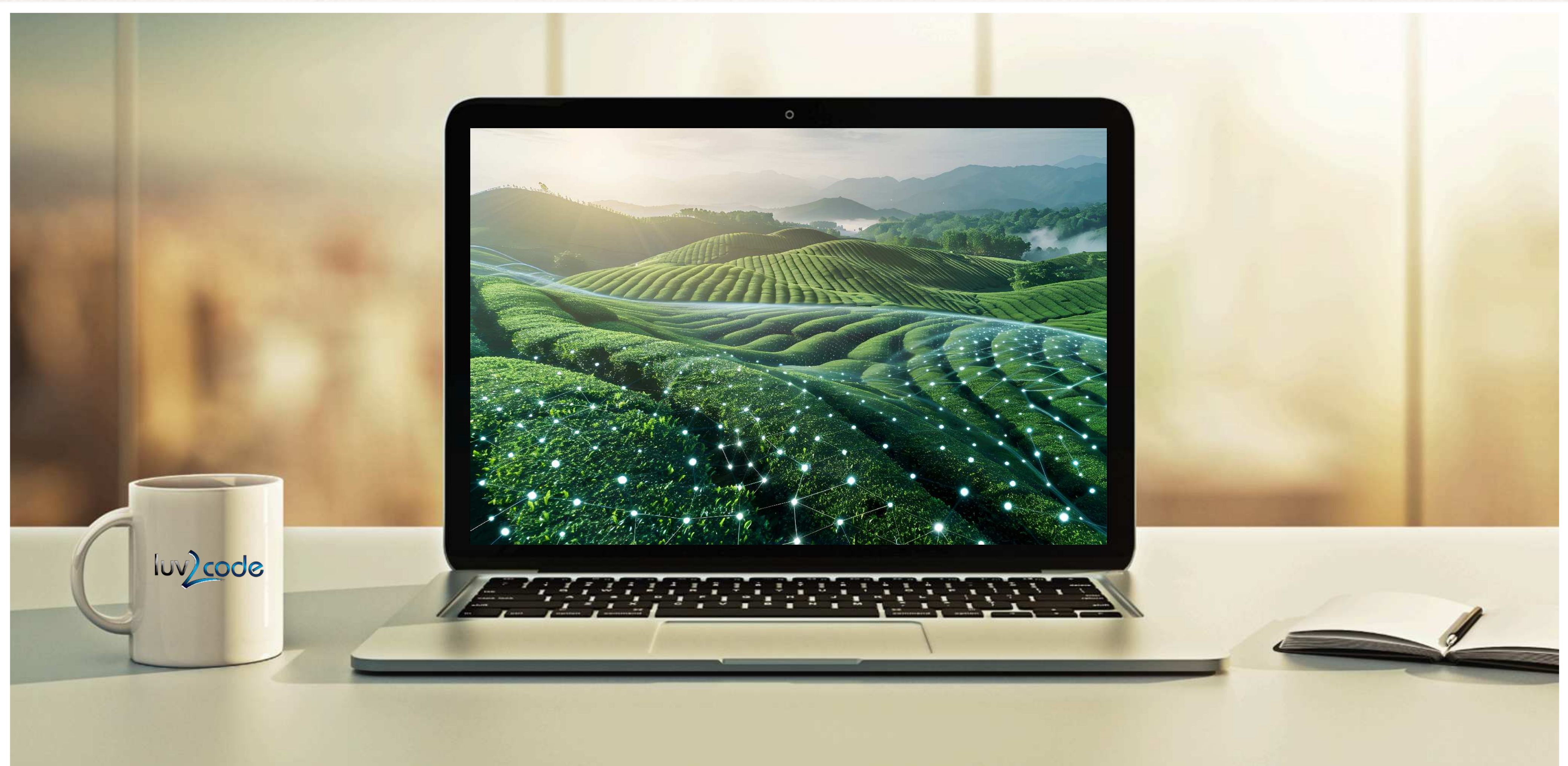
    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }
}
```

2. Create JWT Service.

- 1. Token Parsing ✓
- 2. Token Validation ✓
- 3. Token Generation ✓

JWT Authentication Filter



Development Process

Step-By-Step

1. Install JWT into our Java Spring Boot Project

2. Create JWT Service.

1. Token Parsing

2. Token Validation

3. Token Generation

3. JWT Security Authentication Filter

1. Intercept every HTTP request.

2. If endpoint require authentication, validate token.

Deeper dive in JWT Auth Filter

- Essentially a security checkpoint for our application.
- Every request gets checked, if the endpoint requires authentication, we valid the JWT attached.
- Extract their credentials
- Verify the token is legitimate and hasn't expired.

JWT Authentication filter

File: JwtAuthenticationFilter.java

```
@Component
public class JwtAuthenticationFilter implements OncePerRequestFilter {

    private final JwtService jwtService;

    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService, @Lazy UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    ...Continues on next slide...

}
```


JWT Authentication filter

File: JwtAuthenticationFilter.java

```
@Override
protected void doFilterInternal(@NonNull HttpServletRequest request, @NonNull HttpServletResponse response,
    @NonNull FilterChain filterChain) throws ServletException, IOException {
    final String authHeader = request.getHeader("Authorization");
    final String jwt;
    final String userEmail;

    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        filterChain.doFilter(request, response);
        return;
    }

    jwt = authHeader.substring(7);
    userEmail = jwtService.extractUsername(jwt);

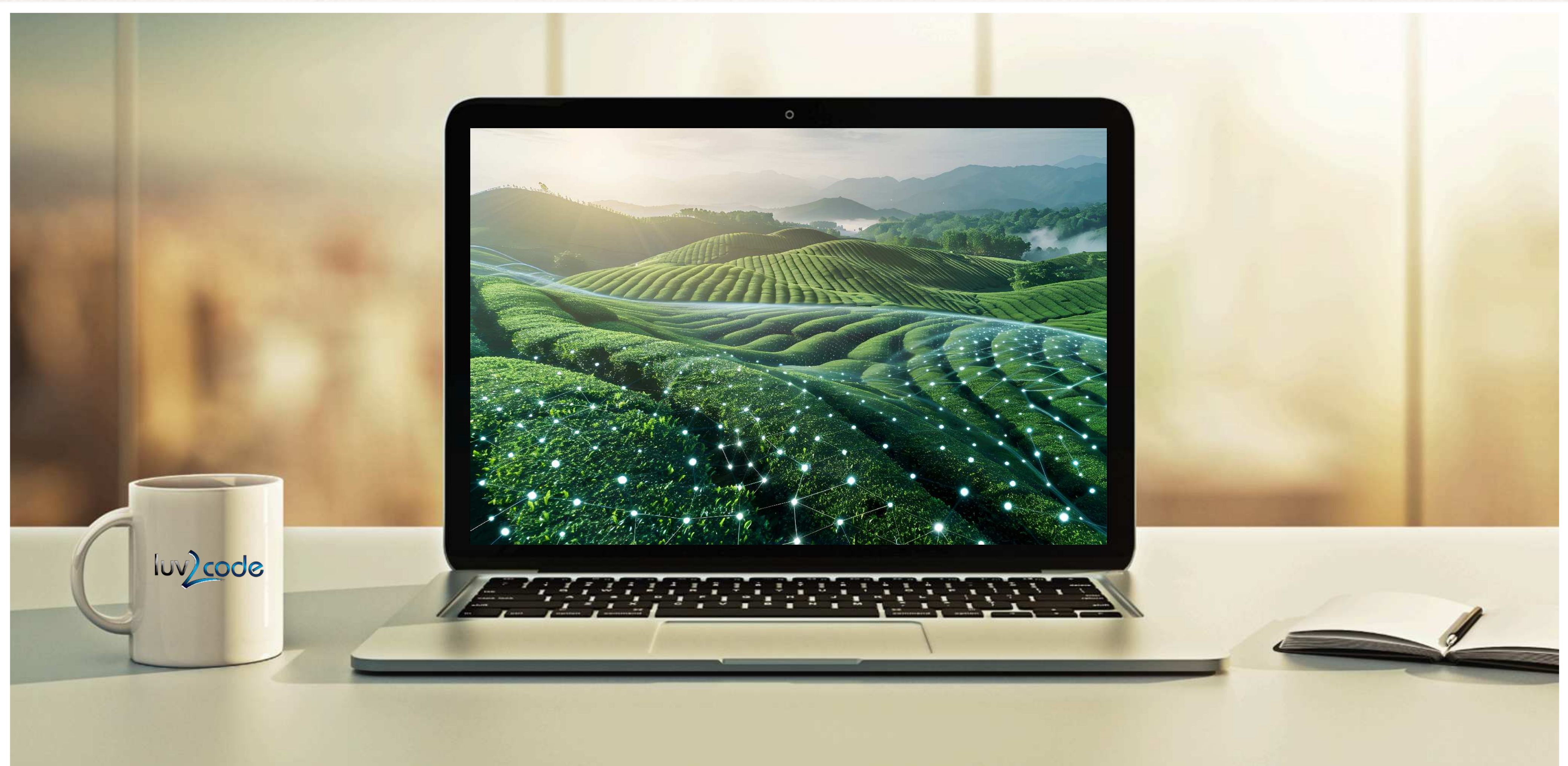
    if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails userDetails = this.userDetailsService.loadUserByUsername(userEmail);

        if (jwtService.isTokenValid(jwt, userDetails)) {
            UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());

            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
        filterChain.doFilter(request, response);
    }
}
```


Security Filter & User Repository



User Repository

- Spring Data JPA repository interface for managing User entities.

File: UserRepository.java

```
@Repository
public interface UserRepository extends CrudRepository<User, Long> {
    Optional<User> findByEmail(String email);
}
```

- @Repository marks the interface a Spring Repository making it responsible for data access operations.
- CrudRepository provides basic CRUD operations. User is the entity and Long is the primary key.
- The function automatically creates a query to find a user by their email.

Security Config

- Setup our Spring Boot Security configurations.

File: SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final UserRepository userRepository;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(UserRepository userRepository,
                          JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.userRepository = userRepository;
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }
}
```

...Continues on next slide...

Security Config

File: SecurityConfig.java

```
@Bean
UserDetailsService userDetailsService() {
    return username -> userRepository.findByEmail(username)
        .orElseThrow(() -> new UsernameNotFoundException("User not found"));
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}

@Bean
public AuthenticationEntryPoint authenticationEntryPoint() {
    return (request, response, ex) -> {
        response.setStatus(HttpStatus.UNAUTHORIZED.value());
        response.setContentType("application/json");
        response.setHeader("WWW-Authenticate", "");
        response.getWriter().write("{\"error\": \"Unauthorized access\"}");
    };
}
```

{bcrypt}<hash>

Security Config

File: SecurityConfig.java

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(configurer ->
        configurer
            .requestMatchers("/swagger-ui/**", "/v3/api-docs/**",
                "/swagger-resources/**", "/webjars/**", "/docs").permitAll());

    http.csrf(csrf -> csrf.disable());

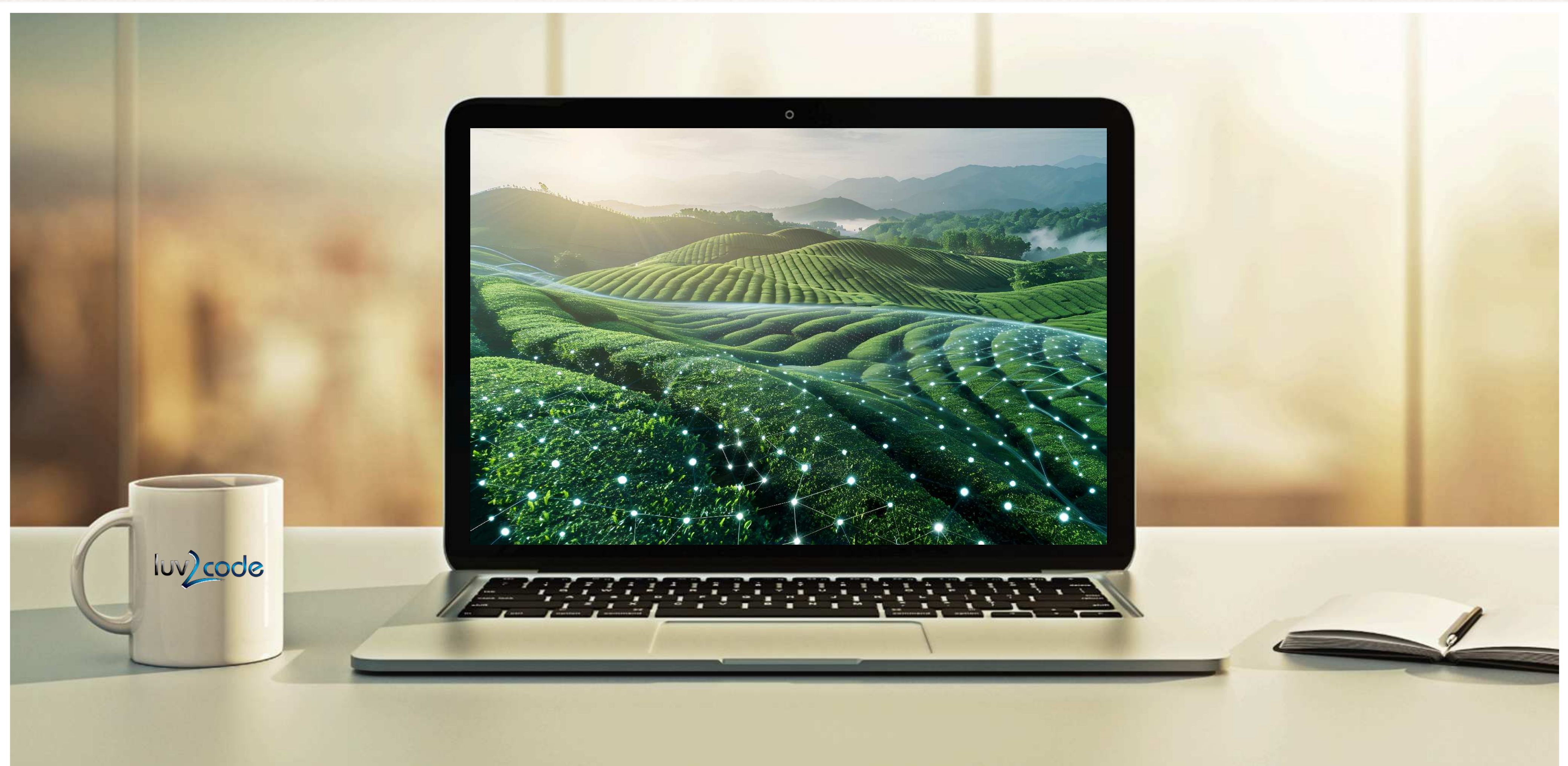
    http.exceptionHandling(exceptionHandling ->
        exceptionHandling
            .authenticationEntryPoint(authenticationEntryPoint()));

    http.sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```


Register User



Development Process

Step-By-Step

1. Create RegisterRequest with data validation
2. Create AuthenticationService interface (register)
 1. Create AuthenticationServiceImpl (register logic)
3. Authentication Controller (register)

Register Request

File: RegisterRequest.java

```
public class RegisterRequest {

    @NotEmpty(message = "First name is mandatory")
    @Size(min = 3, max = 30, message = "First name must be at least 3 characters long")
    private String firstName;

    @NotEmpty(message = "Last name is mandatory")
    @Size(min = 3, max = 30, message = "Last name must be at least 3 characters long")
    private String lastName;

    @NotEmpty(message = "Email is mandatory")
    @Email(message = "Invalid email format")
    private String email;

    @NotEmpty(message = "Password is mandatory")
    @Size(min = 5, max = 30, message = "Password must be at least 5 characters long")
    private String password;

    public RegisterRequest(String firstName, String lastName, String email, String password) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.password = password;
    }

    <...getters & setters...>
}
```


Development Process

Step-By-Step

1. Create RegisterRequest with data validation

2. Create AuthenticationService interface (register)

1. Create AuthenticationServiceImpl (register logic)

3. Authentication Controller (register)

AuthenticationService

File: AuthenticationService.java

```
public interface AuthenticationService {  
    void register(RegisterRequest input) throws Exception;  
}
```

- Only one function for now that we need to implement in our AuthenticationServiceImpl class.

AuthenticationServiceImpl

File: AuthenticationServiceImpl.java

```
@Service
public class AuthenticationServiceImpl implements
    AuthenticationService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthenticationServiceImpl(UserRepository userRepository,
        PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    @Transactional
    public void register(RegisterRequest input) throws Exception {
        if (isEmailTaken(input.getEmail())) {
            throw new Exception("Email already taken");
        }
        User user = buildNewUser(input);
        userRepository.save(user);
    }

    private boolean isEmailTaken(String email) {
        return userRepository.findByEmail(email).isPresent();
    }

    private User buildNewUser(RegisterRequest input) {
        User user = new User();
        user.setId(0);
        user.setFirstName(input.getFirstName());
        user.setLastName(input.getLastName());
        user.setEmail(input.getEmail());
        user.setPassword(passwordEncoder.encode(
            input.getPassword()
        ));
        user.setAuthorities(initialAuthority());
        return user;
    }

    private List<Authority> initialAuthority() {
        boolean isFirstUser = userRepository.count() == 0;
        List<Authority> authorities = new ArrayList<>();
        authorities.add(new Authority("ROLE_EMPLOYEE"));
        if (isFirstUser) {
            authorities.add(new Authority("ROLE_ADMIN"));
        }
        return authorities;
    }
}
```


Development Process

Step-By-Step

1. Create RegisterRequest with data validation
2. Create AuthenticationService interface (register)
 1. Create AuthenticationServiceImpl (register logic)
3. Authentication Controller (register)

AuthenticationController

File: AuthenticationController.java

```
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication REST API Endpoints", description = "Operations related to register & login")
public class AuthenticationController {

    private final AuthenticationService authenticationService;

    public AuthenticationController(AuthenticationService authenticationService) {
        this.authenticationService = authenticationService;
    }

    @Operation(summary = "Register a user", description = "Create new user in database")
    @ResponseStatus(HttpStatus.CREATED)
    @PostMapping("/register")
    public void register(@Valid @RequestBody RegisterRequest registerRequest) throws Exception {
        authenticationService.register(registerRequest);
    }
}
```


Security Config

File: SecurityConfig.java

Add this path to be permitted

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(configurer ->
        configurer
            .requestMatchers("/api/auth/**", "/swagger-ui/**", "/v3/api-docs/**",
                "/swagger-resources/**", "/webjars/**", "/docs").permitAll());

    http.csrf(csrf -> csrf.disable());

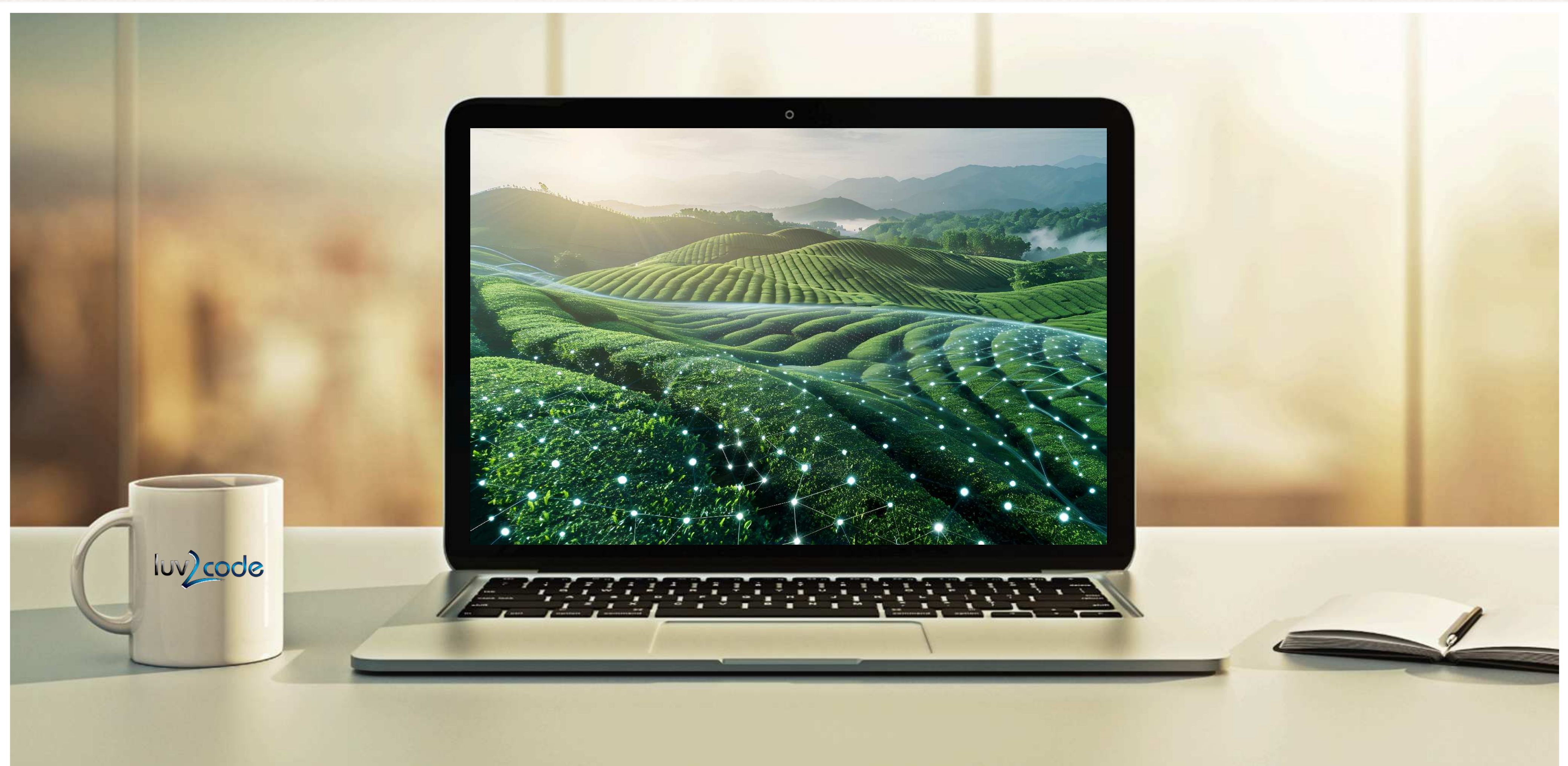
    http.exceptionHandling(exceptionHandling ->
        exceptionHandling
            .authenticationEntryPoint(authenticationEntryPoint()));

    http.sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```


Authenticate User



Development Process

Step-By-Step

1. Create AuthenticateRequest with data validation and AuthenticateResponse

2. Add login to AuthenticationService interface

1. Add login to AuthenticationServiceImpl

3. Add Login to Authentication Controller

Authenticate Request

File: AuthenticationRequest.java

```
public class AuthenticationRequest {  
  
    @NotEmpty(message = "Email is mandatory")  
    @Email(message = "Invalid email format")  
    private String email;  
  
    @NotEmpty(message = "Password is mandatory")  
    @Size(min = 5, max = 30, message = "Password must be at least 5 characters long")  
    private String password;  
  
    public String getEmail() {  
        return email;  
    }  
  
    public String getPassword() {  
        return password;  
    }  
}
```


Authenticate Response

File: AuthenticationResponse.java

```
public class AuthenticationResponse {  
  
    private String token;  
  
    public AuthenticationResponse(String token) {  
        this.token = token;  
    }  
  
    public void setToken(String token) {  
        this.token = token;  
    }  
}
```


Development Process

Step-By-Step

1. Create AuthenticateRequest with data validation and AuthenticateResponse

2. Add login to AuthenticationService interface

1. Add login to AuthenticationServiceImpl

3. Add Login to Authentication Controller

AuthenticationService

File: AuthenticationService.java

```
public interface AuthenticationService {  
    void register(RegisterRequest input) throws Exception;  
    AuthenticationResponse login(AuthenticationRequest request);  
}
```

- Only one new function for now that we need to implement in our AuthenticationServiceImpl class.

AuthenticationServiceImpl

File: AuthenticationServiceImpl.java

```
@Override
@Transactional(readOnly = true)
public AuthenticationResponse login(AuthenticationRequest request) {

    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
    );

    User user = userRepository.findByEmail(request.getEmail())
        .orElseThrow(() -> new IllegalArgumentException("Invalid email or password"));

    String jwtToken = jwtService.generateToken(new HashMap<>(), user);

    return new AuthenticationResponse(jwtToken);
}
```


Development Process

Step-By-Step

1. Create AuthenticateRequest with data validation and AuthenticateResponse
2. Add login to AuthenticationService interface
 1. Add login to AuthenticationServiceImpl
3. Add Login to Authentication Controller

AuthenticationController

File: AuthenticationController.java

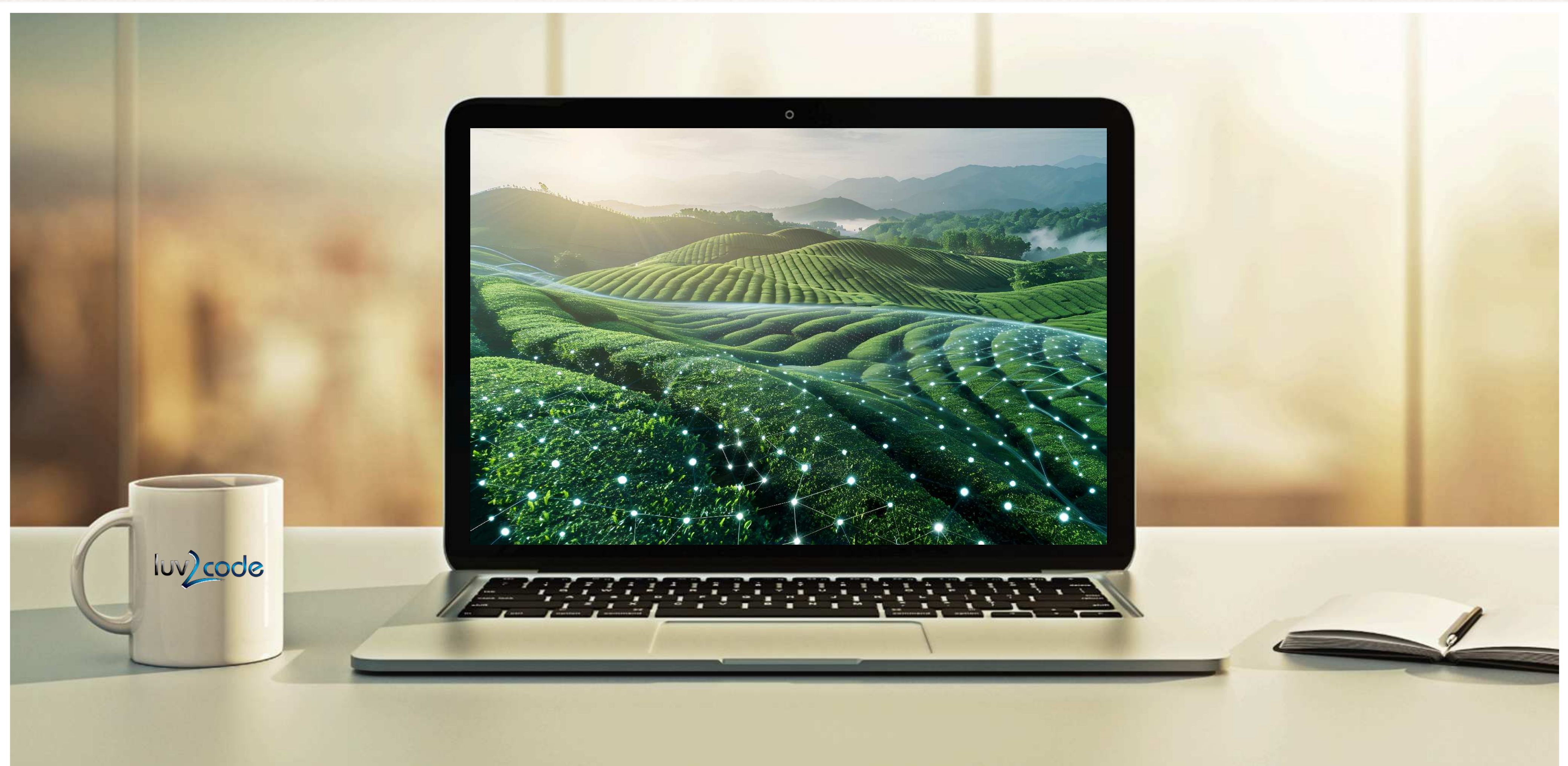
```
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication REST API Endpoints", description = "Operations related to register & login")
public class AuthenticationController {

    private final AuthenticationService authenticationService;

    public AuthenticationController(AuthenticationService authenticationService) {
        this.authenticationService = authenticationService;
    }

    @Operation(summary = "Login a user", description = "Submit email & password to authenticate user.")
    @ResponseStatus(HttpStatus.OK)
    @PostMapping("/login")
    public AuthenticationResponse login(@Valid @RequestBody AuthenticationRequest authRequest) throws Exception {
        authenticationService.login(authRequest);
    }
}
```


Get User Info & Swagger Header



Development Process

Step-By-Step

1. Create UserService interface

1. Create UserServiceImpl (business logic)

2. User Service Controller (getting the user info)

3. Swagger Configurations for headers

4. Create UserResponse

UserService

File: UserService.java

```
public interface UserService {  
    User getUser() ;  
}
```

- Only one function for now that we need to implement in our UserServiceImpl class.

UserServiceImpl

File: UserServiceImpl.java

```
@Service
public class UserServiceImpl implements UserService {

    private final UserRepository userRepository;

    public UserServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    @Transactional(readonly = true)
    public User getUserInfo() throws Exception {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

        if (authentication == null || !authentication.isAuthenticated()) ||
            authentication.getPrincipal().equals("anonymousUser")) {

            throw new AccessDeniedException("Authentication required");

            return (User) authentication.getPrinciple();
        }
    }
}
```


UserController

File: UserController.java

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "User REST API Endpoints", description = "Operations related to info about current user")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/info")
    public User getUserInfo() {
        return userService.getUserInfo();
    }
}
```


Is there something missing in Swagger?

User REST API Endpoints Operations related to info about current user

GET /api/users/info

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/users/info' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/users/info
```

Server response

- No where to pass in JWT for validation.

Security Config

File: SecurityConfig.java

All other paths need authentication.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(configurer ->
        configurer
            .requestMatchers("/api/auth/**", "/swagger-ui/**", "/v3/api-docs/**",
                "/swagger-resources/**", "/webjars/**", "/docs").permitAll()
            .anyRequest().authenticated());

    http.csrf(csrf -> csrf.disable());

    http.exceptionHandling(exceptionHandling ->
        exceptionHandling
            .authenticationEntryPoint(authenticationEntryPoint()));

    http.sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

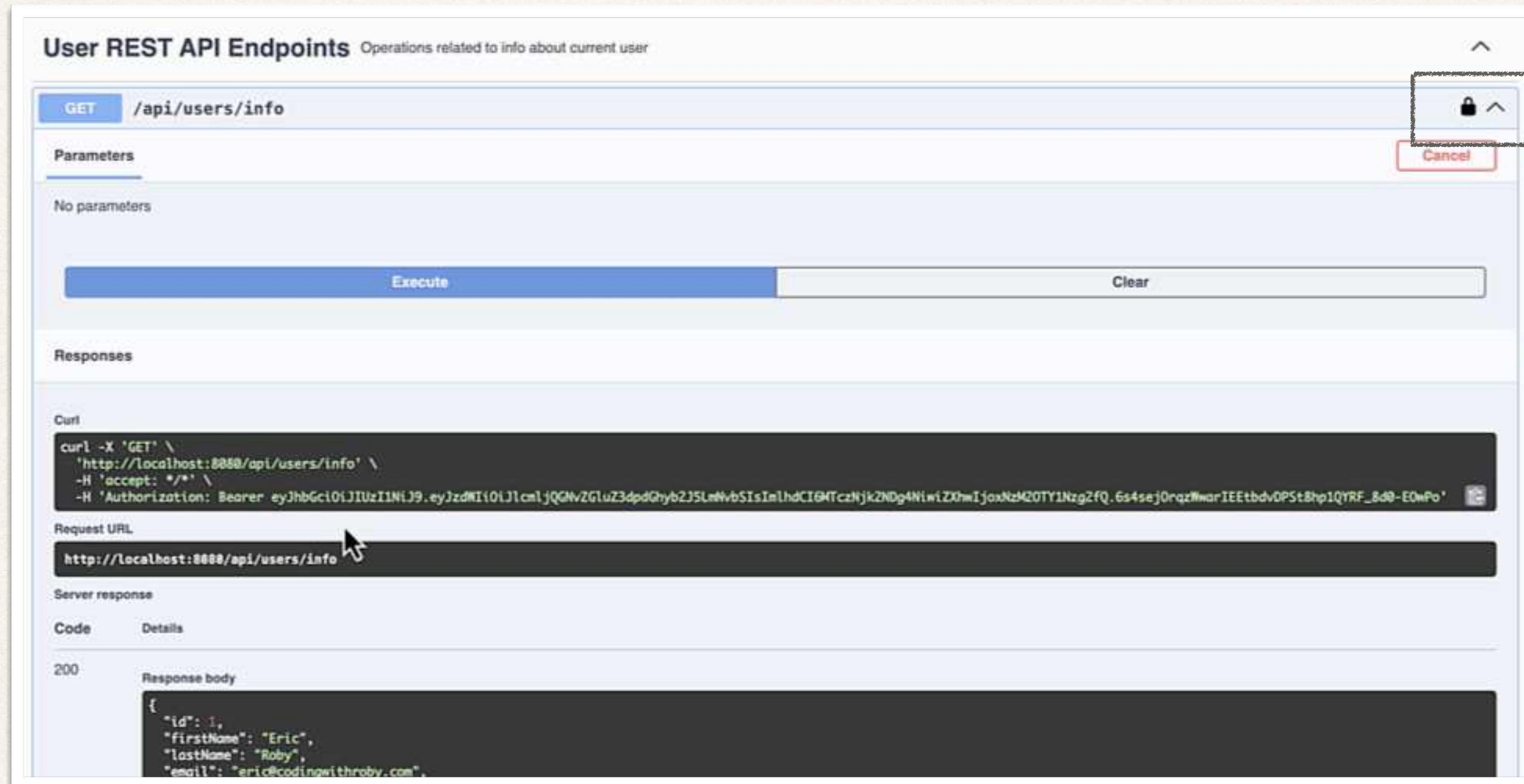
    return http.build();
}
```


Swagger Config

File: SwaggerConfig.java

```
@Configuration
@OpenAPIDefinition(
    info = @Info(title = "API Documentation", version = "v1"),
    security = @SecurityRequirement(name = "bearerAuth")
)
@SecurityScheme(
    name = "bearerAuth",
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT"
)
public class SwaggerConfig {
    // Leave this blank
}
```


Now we can add JWTs to Swagger



Add JWT here
from login

Return an entire user

```
Response body
{
  "id": 1,
  "firstName": "Eric",
  "lastName": "Roby",
  "email": "eric@codingwithroby.com",
  "password": "$2a$10$e0RKtbIgt.CdftPLXd9kGuxKg5DKaFy4F4p4wkJOmHEzQWX.U5n2",
  "authorities": [
    {
      "authority": null
    },
    {
      "authority": null
    }
  ],
  "enabled": true,
  "credentialsNonExpired": true,
  "username": "eric@codingwithroby.com",
  "accountNonExpired": true,
  "accountNonLocked": true
}
```

- Need to return only the items the user needs.

User Response

File: UserResponse.java

```
public class UserResponse {  
  
    private long id;  
    private String fullName;  
    private String email;  
    private List<Authority> authorities;  
  
    public UserResponse(long id, String fullName, String email, List<Authority> authorities) {  
        this.id = id;  
        this.fullName = fullName;  
        this.email = email;  
        this.authorities = authorities;  
    }  
  
    <getters & setters>  
}
```


UserService

File: UserService.java

```
public interface UserService {  
    UserResponse getUser();  
}
```


UserServiceImpl

File: UserServiceImpl.java

```
@Override
@Transactional(readOnly = true)
public UserResponse getUserInfo() throws Exception {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    if (authentication == null || !authentication.isAuthenticated()) ||
        authentication.getPrincipal().equals("anonymousUser")) {

        throw new AccessDeniedException("Authentication required");

        User user = (User) authentication.getPrincipal();

        return new UserResponse(
            user.getId(),
            user.getFirstName() + " " + user.getLastName(),
            user.getEmail(),
            user.getAuthorities().stream().map(auth -> (Authority) auth).toList());
    }
}
```


UserController

File: UserController.java

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "User REST API Endpoints", description = "Operations related to info about current user")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

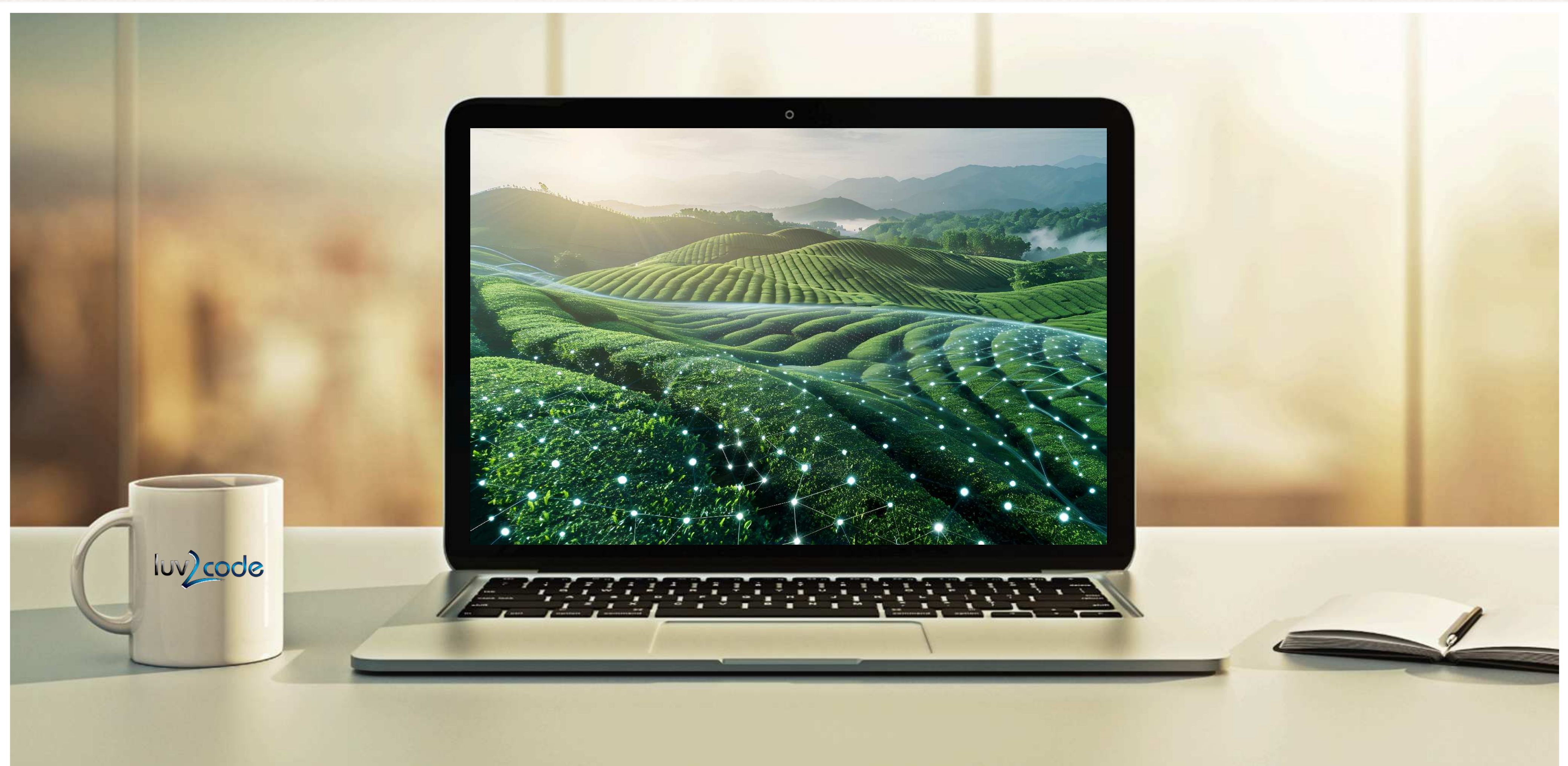
    @GetMapping("/info")
    public UserResponse getUserInfo() {
        return userService.getUserInfo();
    }
}
```


Return a User Response

```
{  
  "id": 1,  
  "fullName": "Eric Roby",  
  "email": "eric@codingwithroby.com",  
  "authorities": [  
    {  
      "authority": null  
    },  
    {  
      "authority": null  
    }  
  ]  
}
```

- Only the items that we need.

Delete User



Development Process

Step-By-Step

1. Add delete to UserService interface

1. Add delete to UserServiceImpl

2. Write custom query to find how many admins are in the application.

3. Add delete to User Controller

UserService

File: UserService.java

```
public interface UserService {  
    UserResponse getUser();  
    void deleteUser();  
}
```

- Implement delete user business logic in our UserServiceImpl class.

UserServiceImpl

File: UserServiceImpl.java

```
@Override
public void deleteUser() {

    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

    if (authentication == null || !authentication.isAuthenticated()) ||
        authentication.getPrincipal().equals("anonymousUser")) {
        throw new AccessDeniedException("Authentication required");
    }

    User user = (User) authentication.getPrincipal();

    if (isLastAdmin(user)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, "Admin cannot delete itself");
    }

    userRepository.delete(user);
}

private boolean isLastAdmin(User user) {
    boolean isAdmin = user.getAuthorities().stream()
        .anyMatch(authority -> "ROLE_ADMIN".equals(authority.getAuthority()));

    if (isAdmin) {
        long adminCount = userRepository.countAdminUsers();
        return adminCount <= 1;
    }

    return false;
}
```

Custom Query

UserRepository

File: UserRepository.java

```
@Repository
public interface UserRepository extends CrudRepository<User, Long> {

    Optional<User> findByEmail(String email);

    @Query("SELECT COUNT(u) FROM User u JOIN u.authorities a WHERE a.authority = 'ROLE_ADMIN'")
    long countAdminUsers();
}
```


UserController

File: UserController.java

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "User REST API Endpoints", description = "Operations related to info about current user")
public class UserController {

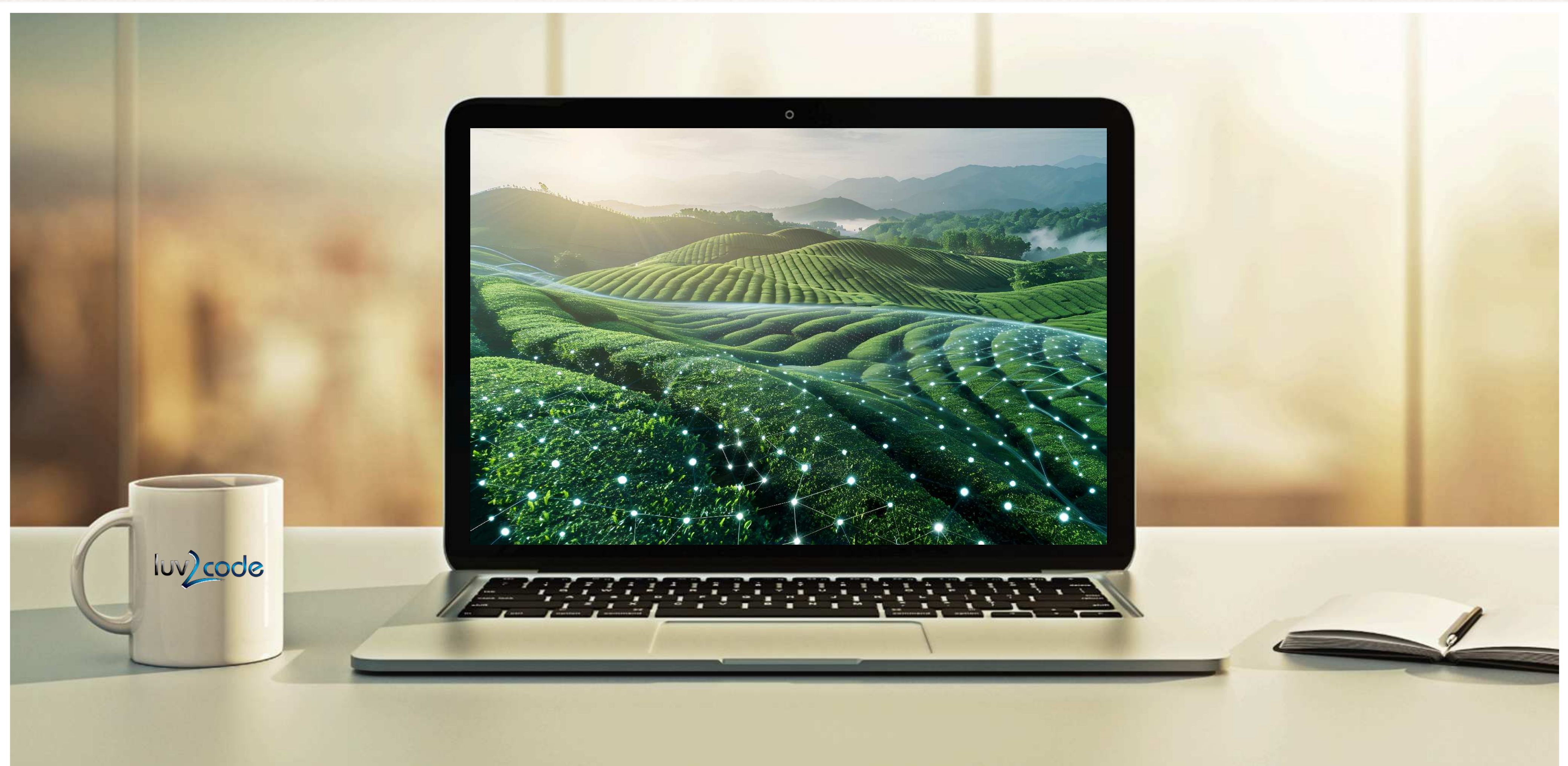
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/info")
    public UserResponse getUserInfo() {
        return userService.getUserInfo();
    }

    @DeleteMapping
    public void deleteUser() {
        userService.deleteUser();
    }
}
```


Exception Handling



Exceptions

- As of now we are not handling exceptions.
- We are returning to the user whatever the exception was which could be a security risk.
- We do not want to give the user more information than they need.

We really want this ...

- Handle the exception and return error as JSON

```
DELETE /api/users/1
```

```
{  
  "status": 403,  
  "message": "Admin cannot delete itself,  
  "timestamp": "1526149650271"  
}
```


Spring REST Exception Handling



Development Process

Step-By-Step

1. Create a custom error response class
2. Create a custom exception class with `@ControllerAdvice`
3. Add a exception handler methods using `@ExceptionHandler`
 1. One for our global Exception
 2. One for our ResponseStatusException

Step 1: Create custom error response class

- The custom error response class will be sent back to client as JSON

- We will define as Java class (POJO)

ExceptionResponses
status : int message : String timeStamp : long
getStatus() : int setStatus(...) : void ...

You can define any custom fields that you want to track

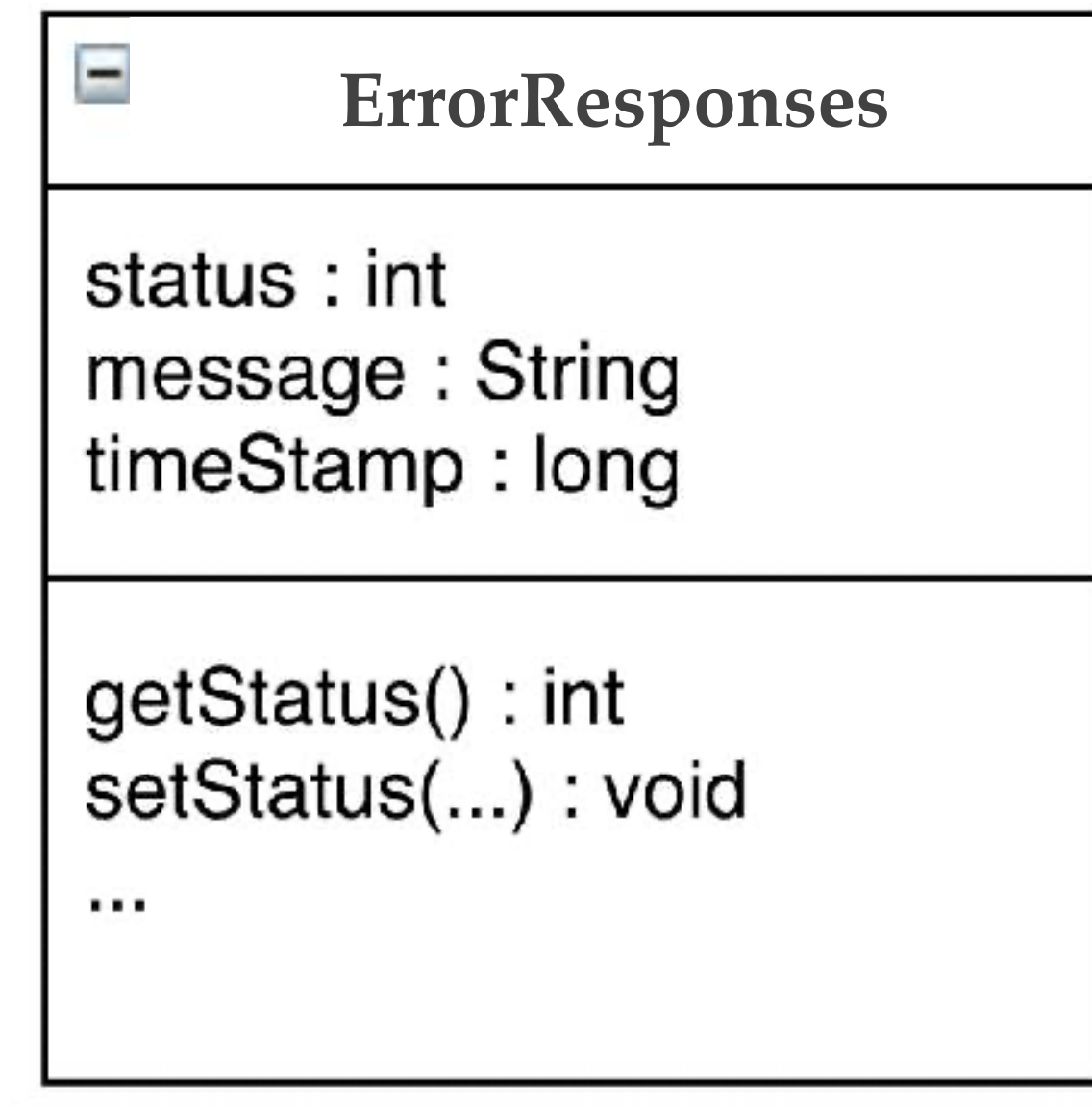
- Jackson will handle converting it to JSON

```
{  
  "status": 403,  
  "message": "Admin cannot delete itself,  
  "timestamp": "1526149650271"  
}
```


Step 1: Create custom error response class

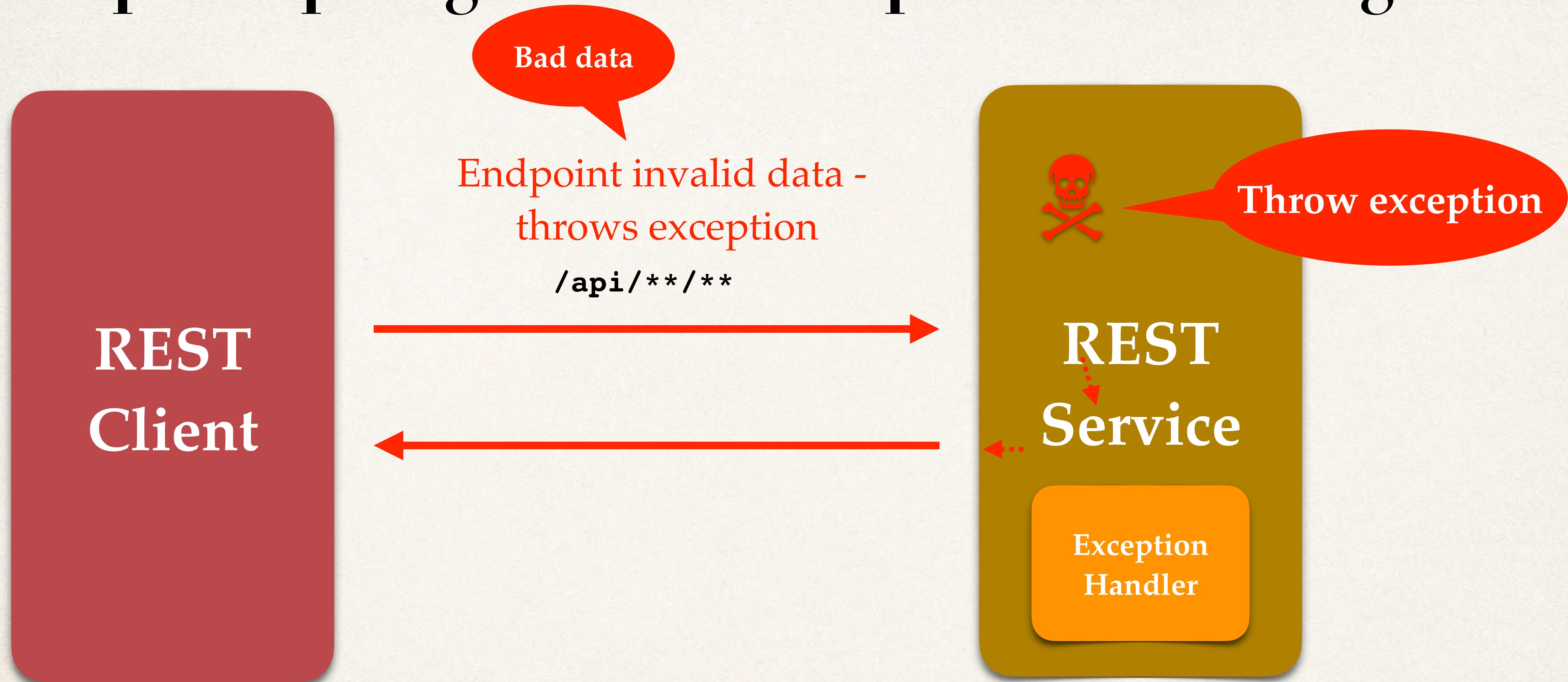
File: ErrorResponse.java

```
public class ErrorResponse {  
  
    private int status;  
    private String message;  
    private long timeStamp;  
  
    // constructors  
  
    // getters / setters  
  
}
```



```
{  
    "status": 403,  
    "message": "Admin cannot delete itself,  
    "timestamp": "1526149650271"  
}
```


Step 2: Spring REST Exception Handling

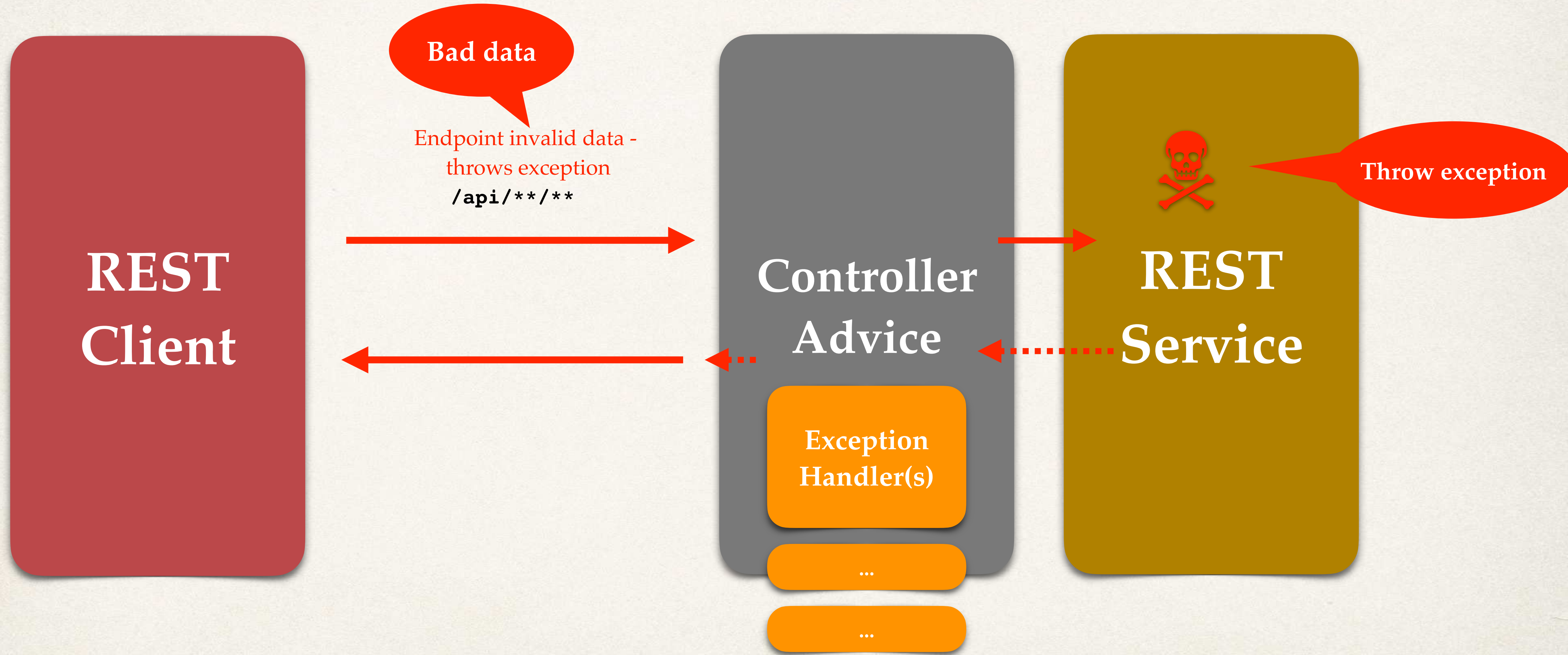


Spring @ControllerAdvice

- @ControllerAdvice is similar to an interceptor / filter
- Pre-process requests to controllers
- Post-process responses to handle exceptions
- Perfect for global exception handling

**Real-time
use of
AOP**

Spring REST Exception Handling



Step 2: Create new @ControllerAdvice

File: ExceptionHandler.java

```
@ControllerAdvice
public class ExceptionHandlers {

    ...

}
```


Step 2: Create new @ControllerAdvice

File: ExceptionHandler.java

```
@ControllerAdvice
public class ExceptionHandlers {

    private ResponseEntity<ExceptionResponses> buildResponseEntity(Exception exc, HttpStatus status) {
        ExceptionResponses error = new ExceptionResponses();
        error.setStatus(status.value());
        error.setMessage(exc.getMessage());
        error.setTimestamp(System.currentTimeMillis());
        return new ResponseEntity<>(error, status);
    }
}
```


Step 3: Add exception handler to @ControllerAdvice

File: ExceptionHandlers.java

Exception Handlers

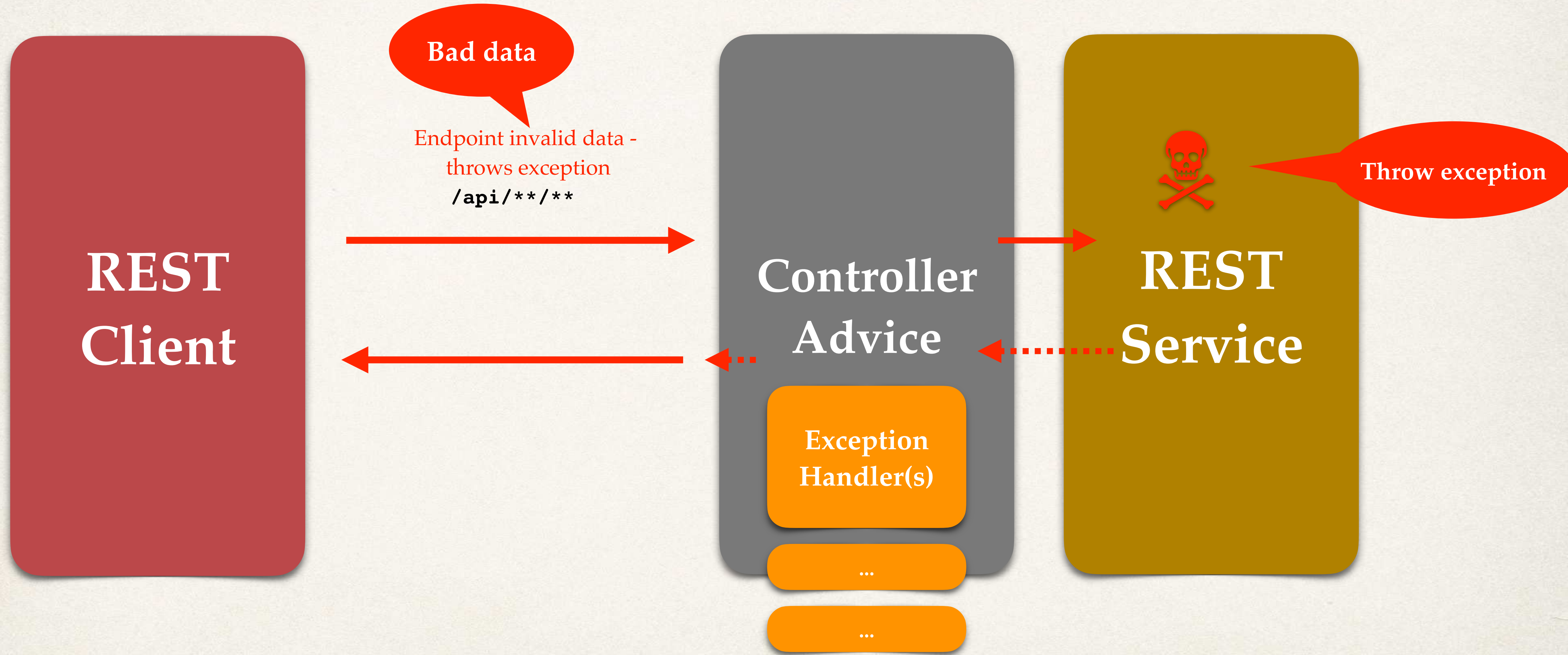
```
@ControllerAdvice
public class ExceptionHandlers {

    @ExceptionHandler(ResponseStatusException.class)
    public ResponseEntity<ExceptionResponses> handleException(ResponseStatusException exc) {
        return buildResponseEntity(exc, HttpStatus.valueOf(exc.getStatusCode().value()));
    }

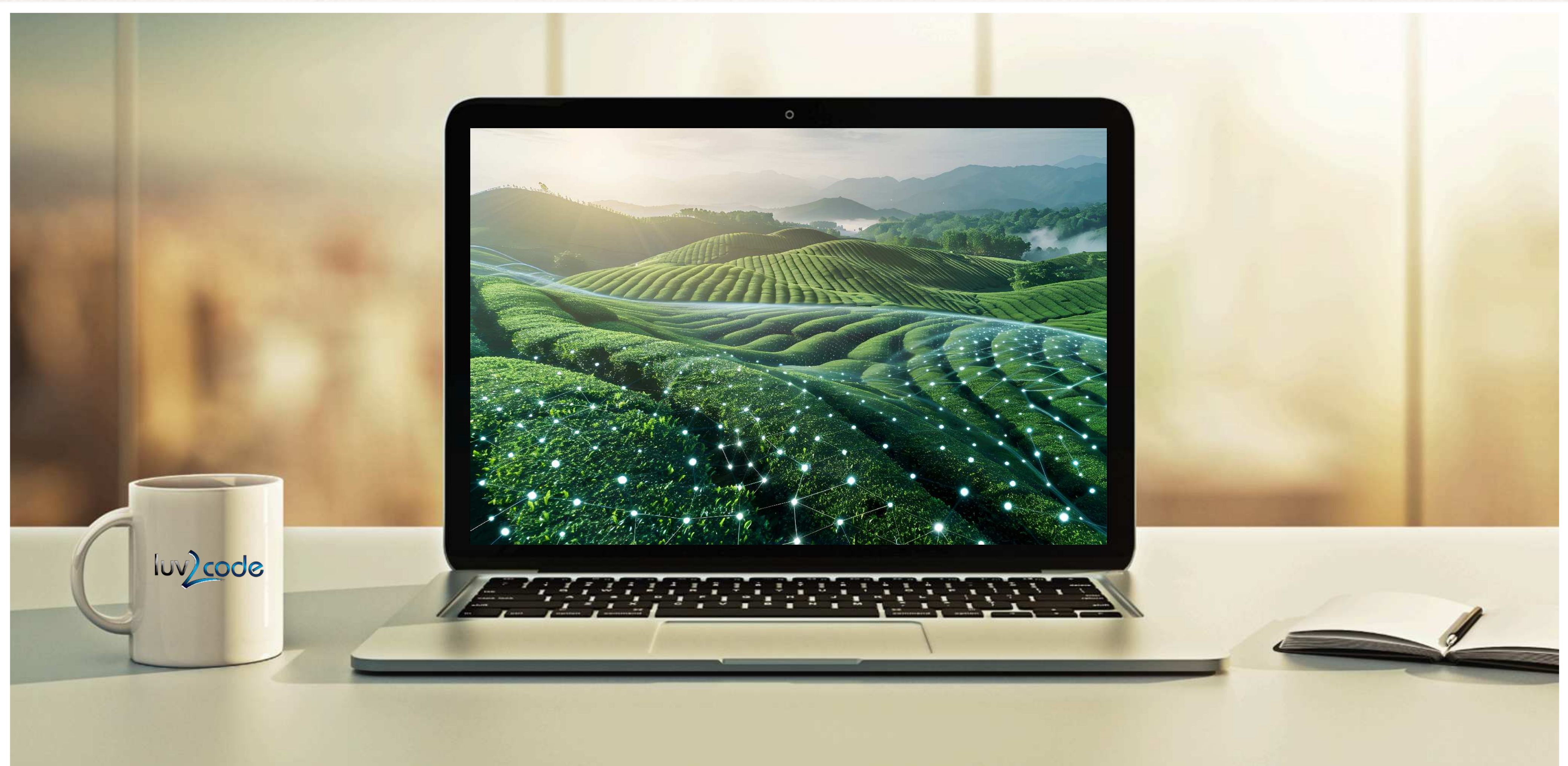
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ExceptionResponses> handleException(Exception exc) {
        return buildResponseEntity(exc, HttpStatus.BAD_REQUEST);
    }

    <...Code from last slide...>
}
```


Spring REST Exception Handling



Utilities



Utilities

- We have two pieces of code using the same snippet wrote twice.
- getUserInfo
- deleteUser

```
Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

if (authentication == null || !authentication.isAuthenticated()) ||
    authentication.getPrincipal().equals("anonymousUser")) {
    throw new AccessDeniedException("Authentication required");
}

User user = (User) authentication.getPrinciple();
```


FindAuthenticatedUser

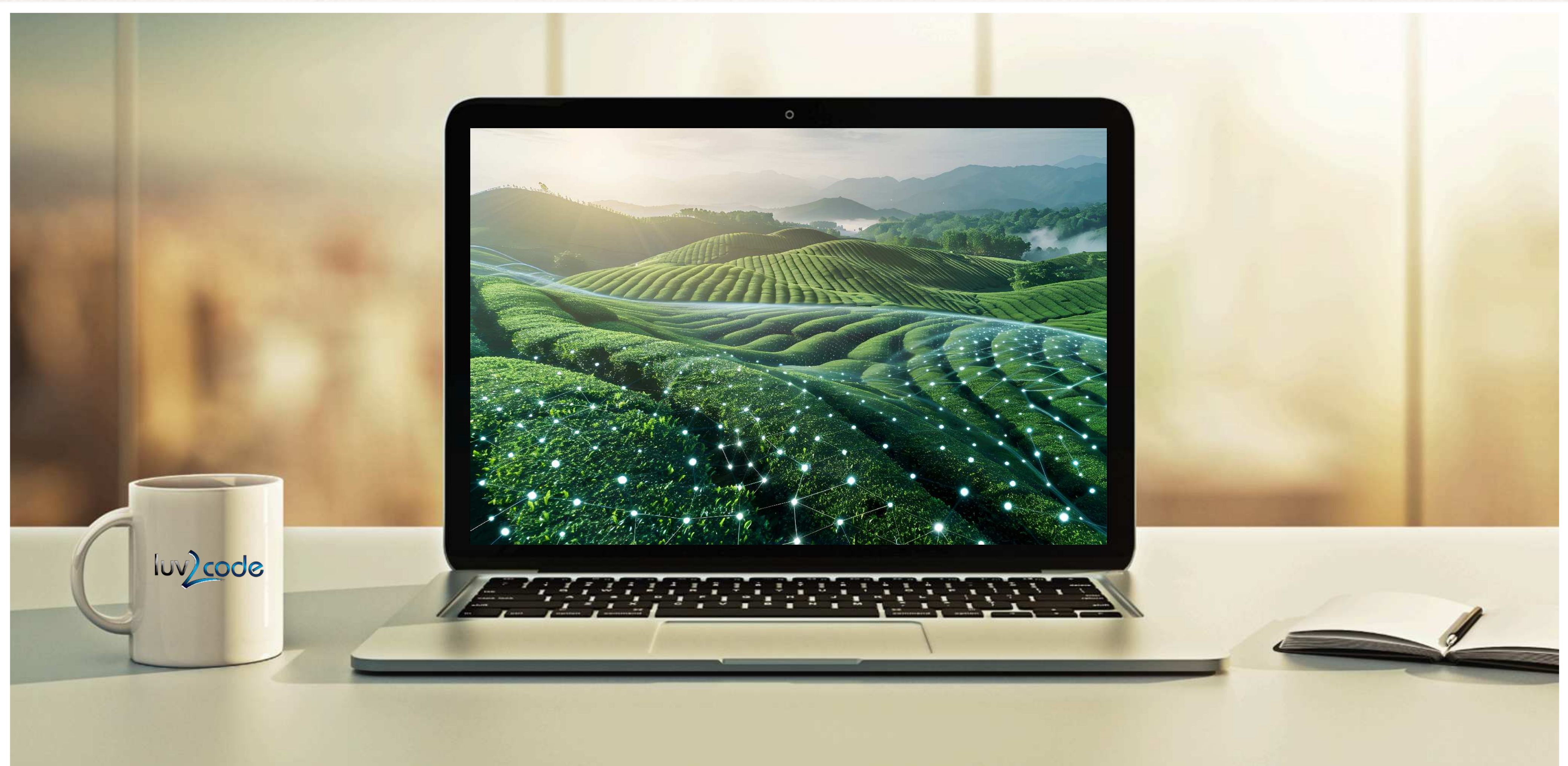
File: FindAuthenticatedUser

```
public interface FindAuthenticatedUser {  
    User getAuthenticatedUser();  
}
```


Utilities

```
public class FindAuthenticatedUserImpl implements FindAuthenticatedUser {  
  
    @Override  
    public User getAuthenticatedUser() {  
  
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();  
  
        if (authentication == null || !authentication.isAuthenticated()) ||  
            authentication.getPrincipal().equals("anonymousUser")) {  
            throw new AccessDeniedException("Authentication required");  
        }  
  
        return (User) authentication.getPrincipal();  
  
    }  
}
```


Update Password



Development Process

Step-By-Step

1. Create PasswordUpdateRequest DTO
2. Add update password to UserService interface
 1. Add update password to UserServiceImpl
3. Add update password to User Controller

PasswordUpdate Request

File: PasswordUpdateRequest.java

```
public class PasswordUpdateRequest {

    @NotEmpty(message = "Old password is mandatory")
    @Size(min = 5, max = 30, message = "Old password must be at least 5 characters long")
    private String oldPassword;

    @NotEmpty(message = "New password is mandatory")
    @Size(min = 5, max = 30, message = "New password must be at least 5 characters long")
    private String newPassword;

    @NotEmpty(message = "New password is mandatory")
    @Size(min = 5, max = 30, message = "New password must be at least 5 characters long")
    private String newPassword2;

    public PasswordUpdateRequest(String oldPassword, String newPassword, String newPassword2) {
        this.oldPassword = oldPassword;
        this.newPassword = newPassword;
        this.newPassword2 = newPassword2;
    }

    <...getters & setters...>
}
```


UserService

File: UserService.java

```
public interface UserService {  
    UserResponse getUser();  
    void deleteUser();  
    Void updatePassword(PasswordUpdateRequest passwordUpdateRequest);  
}
```

- Implement update password business logic in our UserServiceImpl class.

Update Password Checks

- Is the oldPassword, correct?
 - We need to check if the oldPassword matches what we have in the database.
- Is newPassword and newPassword2 the same?
 - We need to validate the user typed in the correct information both times.
- Is newPassword not the same as oldPassword?
 - We only want to implement this logic if the user actually adding a new password.

UserServiceImpl

File: UserServiceImpl.java

```
private boolean isOldPasswordCorrect(String currentPassword, String oldPassword) {  
    return passwordEncoder.matches(oldPassword, currentPassword);  
}
```

Old password
correct?

```
private boolean isNewPasswordConfirmed(String newPassword, String newPasswordConfirmation) {  
    return newPassword.equals(newPasswordConfirmation);  
}
```

Passwords
match?

```
private boolean isNewPasswordDifferent(String oldPassword, String newPassword) {  
    return !oldPassword.equals(newPassword);  
}
```

New
password diff from
old password?

UserServiceImpl

File: UserServiceImpl.java

```
@Override
@Transactional
public void updatePassword(PasswordUpdateRequest passwordUpdateRequest) {
    User user = findAuthenticatedUser.getAuthenticatedUser();

    if (!isOldPasswordCorrect(user.getPassword(), passwordUpdateRequest.getOldPassword())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Current password is incorrect");
    }

    if (!isNewPasswordConfirmed(passwordUpdateRequest.getNewPassword(),
        passwordUpdateRequest.getNewPassword2())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "New passwords do not match");
    }

    if (!isNewPasswordDifferent(passwordUpdateRequest.getOldPassword(),
        passwordUpdateRequest.getNewPassword())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "Old and new passwords must be different");
    }

    user.setPassword(passwordEncoder.encode(passwordUpdateRequest.getNewPassword()));
    userRepository.save(user);
}
```

Old password
correct?

Passwords
match?

New
password diff from
old password?

UserController

File: UserController.java

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "User REST API Endpoints", description = "Operations related to info about current user")
public class UserController {

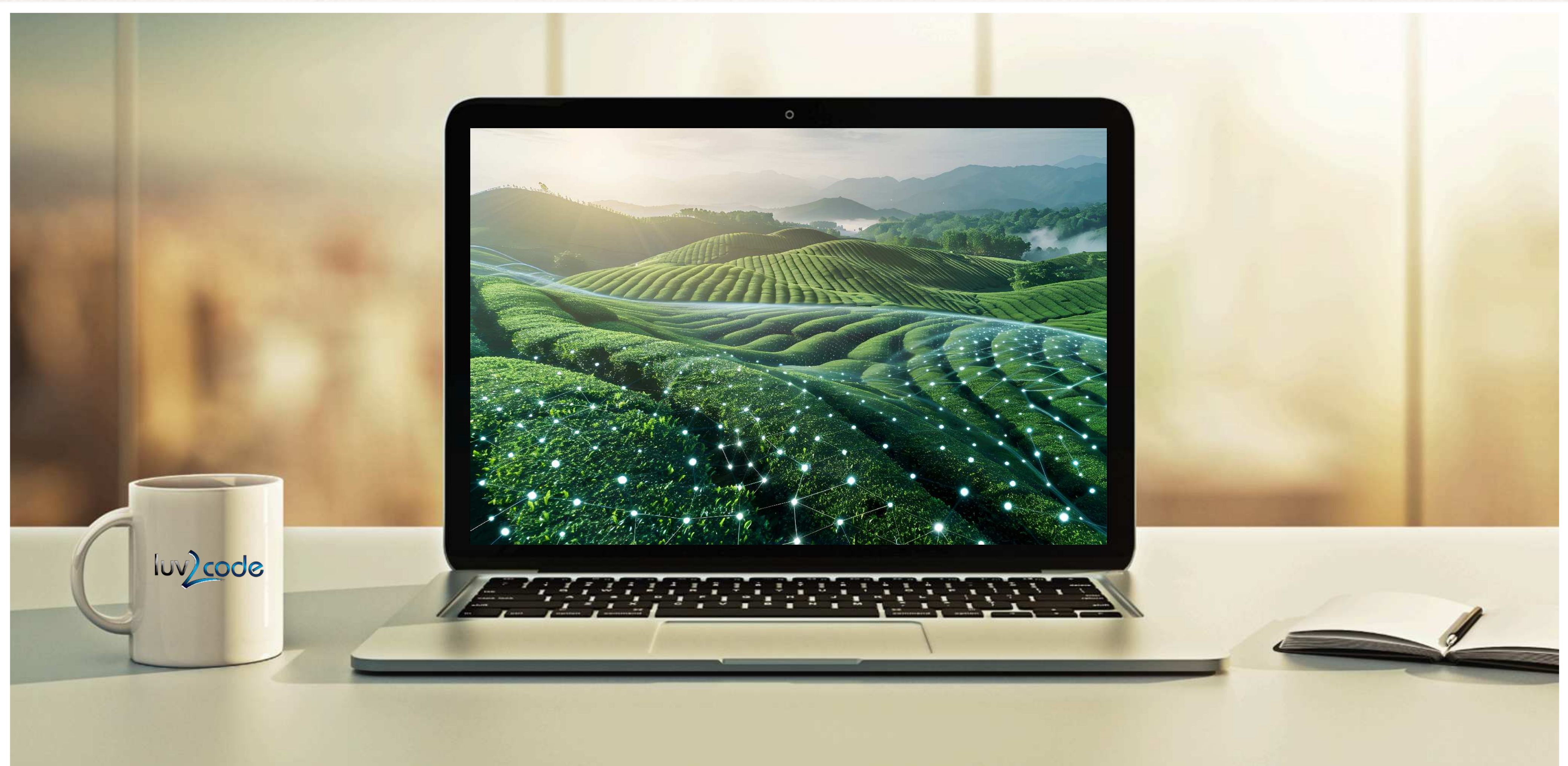
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    <... Other endpoints ...>

    @PutMapping("/password")
    public void passwordUpdate(@Valid @RequestBody PasswordUpdateRequest passwordUpdateRequest)
        throws Exception {
        userService.updatePassword(passwordUpdateRequest);
    }
}
```


Todos



Development Process

Step-By-Step

1. Create the TodoResponse & TodoRequest DTO objects
2. Create TodoRepository for CRUD operations.
3. Create TodoService interface (create todo)
 1. Create TodoServiceImpl for business logic (create todo)
4. Create TodoController for creating a todo

TodoResponse DTO

File: TodoResponse.java

```
public class TodoResponse {  
  
    private long id;  
  
    private String title;  
  
    private String description;  
  
    private int priority;  
  
    private boolean complete;  
  
    public TodoResponse(long id, String title, String description, int priority, boolean complete) {  
        this.id = id;  
        this.title = title;  
        this.description = description;  
        this.priority = priority;  
        this.complete = complete;  
    }  
  
    <...getters & setters...>  
}
```


TodoRequest DTO

File: TodoRequest.java

```
public class TodoRequest {

    @NotEmpty(message = "Title is mandatory")
    @Size(min = 3, max = 30, message = "Title must be at least 3 characters long")
    private String title;

    @NotEmpty(message = "Description is mandatory")
    @Size(min = 3, max = 30, message = "Description must be at least 3 characters long")
    private String description;

    @Min(1)
    @Max(5)
    private int priority;

    public TodoRequest(String title, String description, int priority) {
        this.title = title;
        this.description = description;
        this.priority = priority;
    }

    <...getters & setters...>
}
```


TodoRepository

File: TodoRepository.java

```
public interface TodoRepository extends CrudRepository<Todo, Long> {  
}
```

- Automatically adds CRUD operations for the Todo entity.

TodoService

File: TodoService.java

```
public interface TodoService {  
    TodoResponse createTodo(TodoRequest todoRequest);  
}
```

- Implement create todo business logic in our TodoServiceImpl class.

TodoServiceImpl

File: TodoServiceImpl.java

```
@Service
public class TodoServiceImpl implements TodoService {

    private final TodoRepository todoRepository;
    private final FindAuthenticatedUser findAuthenticatedUser;

    public TodoServiceImpl(TodoRepository todoRepository, FindAuthenticatedUser findAuthenticatedUser) {
        this.todoRepository = todoRepository;
        this.findAuthenticatedUser = findAuthenticatedUser;
    }

    @Override
    @Transactional
    public TodoResponse createTodo(TodoRequest todoRequest) {
        User currentUser = findAuthenticatedUser.getAuthenticatedUser();

        Todo todo = <Convert request into Todo Entity>

        Todo savedTodo = todoRepository.save(todo);

        TodoResponse todoResponse = <Convert savedTodo into a TodoRequest>

        return todoResponse;
    }
}
```


TodoController

File: TodoController.java

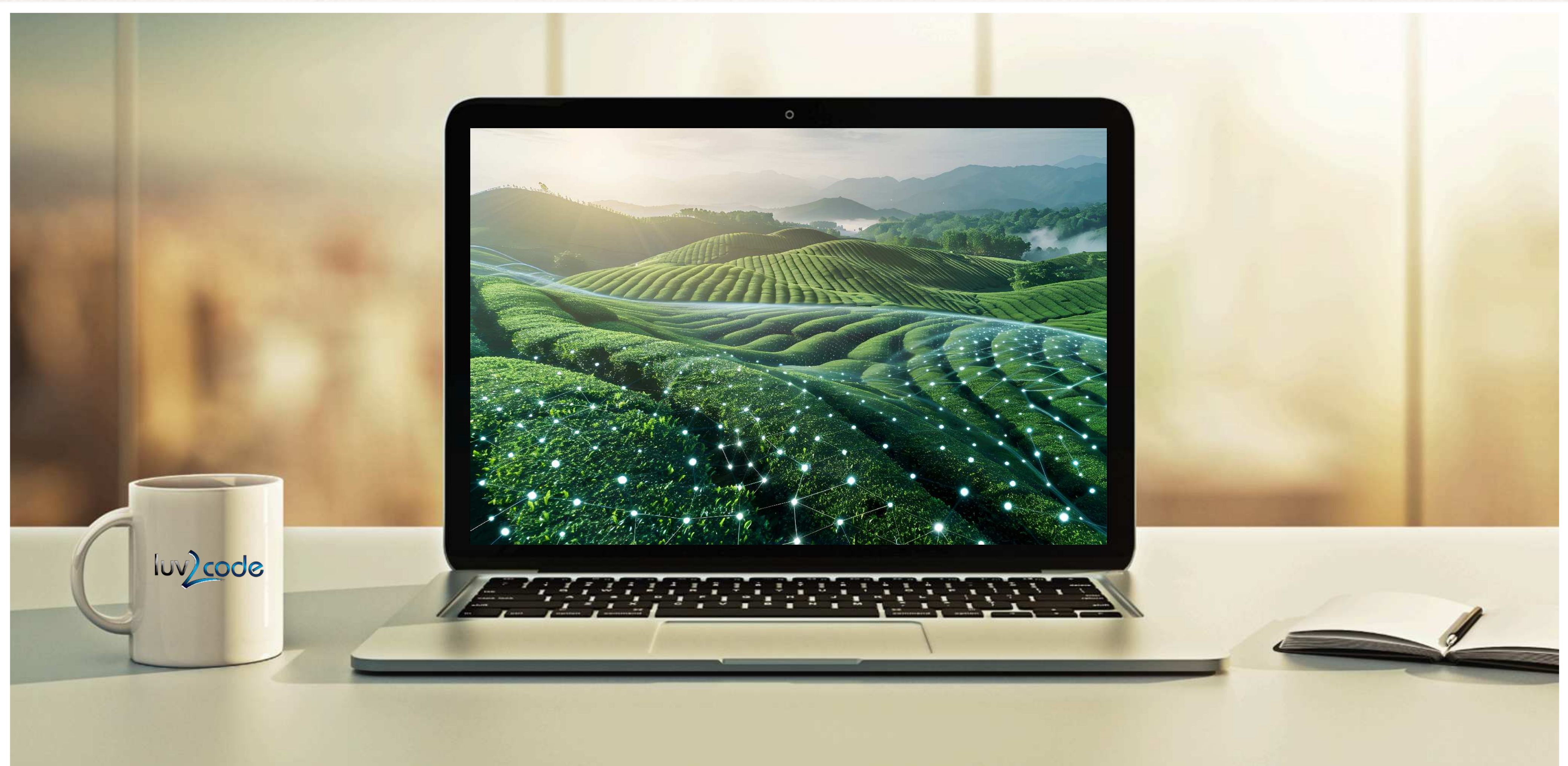
```
@RestController
@RequestMapping("/api/todos")
@Tag(name = "Todo REST API Endpoints", description = "Operations for managing user todos")
public class TodoController {

    private final TodoService todoService;

    public TodoController(TodoService todoService) {
        this.todoService = todoService;
    }

    @Operation(summary="Create todo for user", description="Create todo for the signed in user")
    @ResponseStatus(HttpStatus.CREATED)
    @PostMapping
    public TodoResponse createTodo(@Valid @RequestBody TodoRequest todoRequest) {
        return todoService.createTodo(todoRequest);
    }
}
```


Todos Fetch



Development Process



1. Update TodoRepository to find all todos based on authenticated user.
2. Update TodoService for getAllTodos
3. Update TodoServiceImpl to add business logic for getAllTodos
4. Update TodoController to fetch all todos.

TodoRepository

File: TodoRepository.java

```
public interface TodoRepository extends CrudRepository<Todo, Long> {  
    List<Todo> findByOwner(User owner);  
}
```

- Find all todos based on the authenticated user.

TodoService

File: TodoService.java

```
public interface TodoService {  
    List<TodoResponse> getAllTodos();  
    TodoResponse createTodo(TodoRequest todoRequest);  
}
```

- Implement getAllTodos business logic in our TodoServiceImpl class.

TodoServiceImpl

File: TodoServiceImpl.java

```
@Service
public class TodoServiceImpl implements TodoService {

    private final TodoRepository todoRepository;
    private final FindAuthenticatedUser findAuthenticatedUser;

    public TodoServiceImpl(TodoRepository todoRepository, FindAuthenticatedUser findAuthenticatedUser) {
        this.todoRepository = todoRepository;
        this.findAuthenticatedUser = findAuthenticatedUser;
    }
```

```
    @Override
    @Transactional(readOnly = true)
    public List<TodoResponse> getAllTodos() {
        User currentUser = findAuthenticatedUser.getAuthenticatedUser();

        return todoRepository.findByOwner(currentUser)
            .stream()
            .map(this::convertToTodoResponse)
            .toList();
    }
```

<...Other Code...>

```
}
```


TodoController

File: TodoController.java

```
@RestController
@RequestMapping("/api/todos")
@Tag(name = "Todo REST API Endpoints", description = "Operations for managing user todos")
public class TodoController {

    private final TodoService todoService;

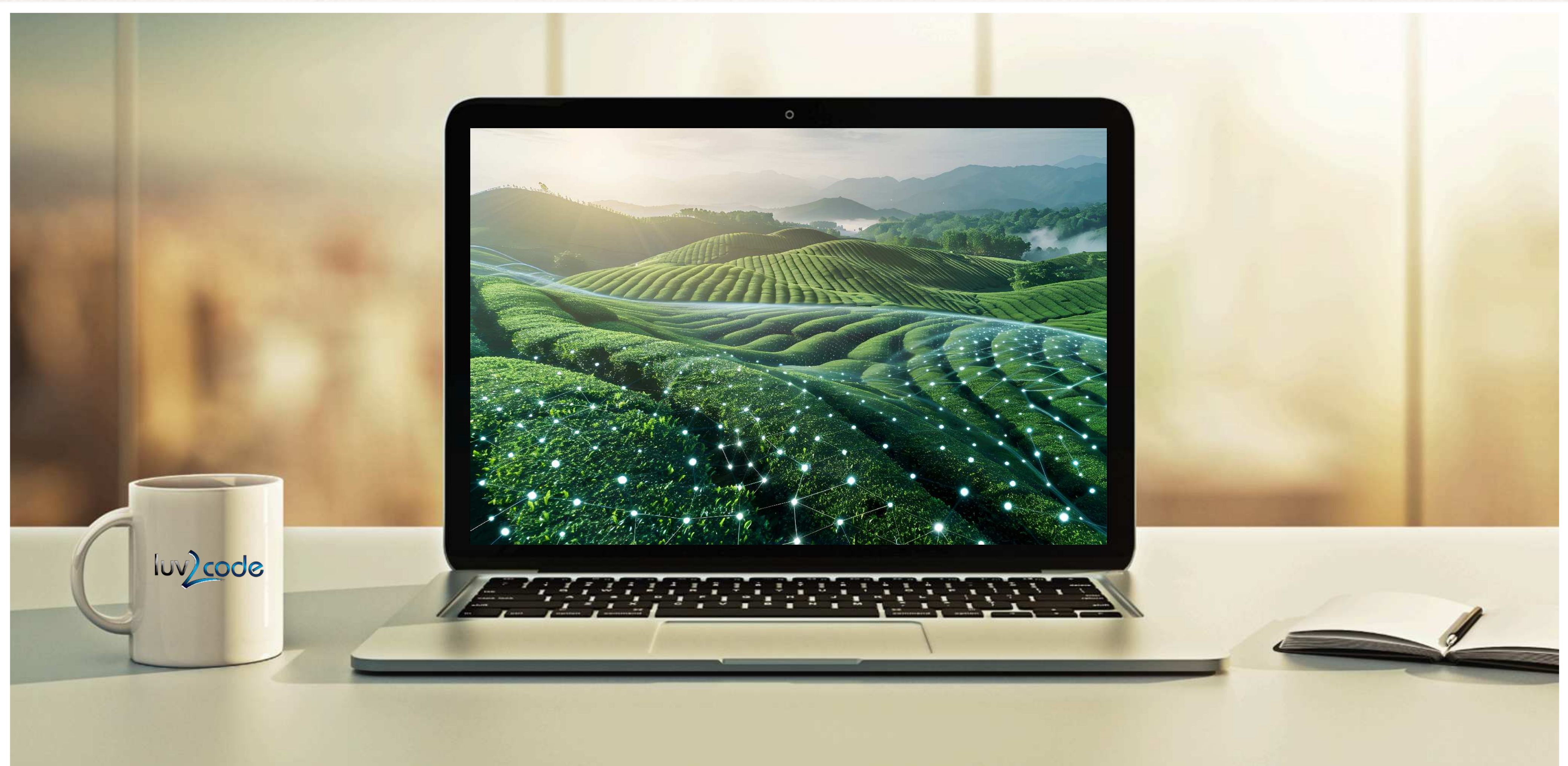
    public TodoController(TodoService todoService) {
        this.todoService = todoService;
    }

    @Operation(summary = "Get all todos for user", description = "Fetch all todos from signed in user")
    @ResponseStatus(HttpStatus.OK)
    @GetMapping
    public List<TodoResponse> getAllTodos() {
        return todoService.getAllTodos();
    }

    <...Other Code...>

}
```


Todos Complete



Development Process



1. Update TodoRepository to find specific todo by ID and owner.
2. Update TodoService for toggleTodoCompletion
3. Update TodoServiceImpl to add business logic for toggleTodoCompletion
4. Update TodoController to have this new endpoint

TodoRepository

File: TodoRepository.java

```
public interface TodoRepository extends CrudRepository<Todo, Long> {  
    List<Todo> findByOwner(User owner);  
    Optional<Todo> findByIdAndOwner(Long id, User owner);  
}
```

- Find specific todo based on id and owner.

TodoService

File: TodoService.java

```
public interface TodoService {  
    List<TodoResponse> getAllTodos();  
    TodoResponse createTodo(TodoRequest todoRequest);  
    TodoResponse toggleTodoCompletion(long id);  
}
```

- Implement toggleTodoCompletion business logic in our TodoServiceImpl class.

TodoServiceImpl

File: TodoServiceImpl.java

```
@Override
@Transactional
public TodoResponse toggleTodoCompletion(long id) {
    User currentUser = findAuthenticatedUser.getAuthenticatedUser();

    Optional<Todo> todo = todoRepository.findByIdAndOwner(id, currentUser);

    if (todo.isEmpty()) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Todo not found");
    }

    todo.get().setComplete(!todo.get().isComplete());
    Todo updatedTodo = todoRepository.save(todo.get());

    return convertToTodoResponse(updatedTodo);
}
```


TodoController

File: TodoController.java

```
@RestController
@RequestMapping("/api/todos")
@Tag(name = "Todo REST API Endpoints", description = "Operations for managing user todos")
public class TodoController {
```

```
    private final TodoService todoService;
```

```
    public TodoController(TodoService todoService) {
        this.todoService = todoService;
    }
```

```
        @Operation(summary = "Update todo for user", description = "Update todo for the signed in user")
        @ResponseStatus(HttpStatus.OK)
        @PutMapping("/{id}")
        public TodoResponse toggleTodoCompletion(@PathVariable @Min(1) long id) {
            return todoService.toggleTodoCompletion(id);
        }
```

<...Other Code...>

```
}
```


Todos Delete



Development Process



1. Update TodoService for delete.
2. Update TodoServiceImpl to add business logic for delete.
3. Update TodoController to have this new endpoint

TodoService

File: TodoService.java

```
public interface TodoService {  
    List<TodoResponse> getAllTodos();  
    TodoResponse createTodo(TodoRequest todoRequest);  
    TodoResponse toggleTodoCompletion(long id);  
    void deleteTodo(long id);  
}
```

- Implement delete business logic in our TodoServiceImpl class.

TodoServiceImpl

File: TodoServiceImpl.java

```
@Override
@Transactional
public void deleteTodo(long id) {
    User currentUser = findAuthenticatedUser.getAuthenticatedUser();

    Optional<Todo> todo = todoRepository.findByIdAndOwner(id, currentUser);

    if (todo.isEmpty()) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Todo not found");
    }
    todoRepository.delete(todo.get());
}
```


TodoController

File: TodoController.java

```
@RestController
@RequestMapping("/api/todos")
@Tag(name = "Todo REST API Endpoints", description = "Operations for managing user todos")
public class TodoController {

    private final TodoService todoService;

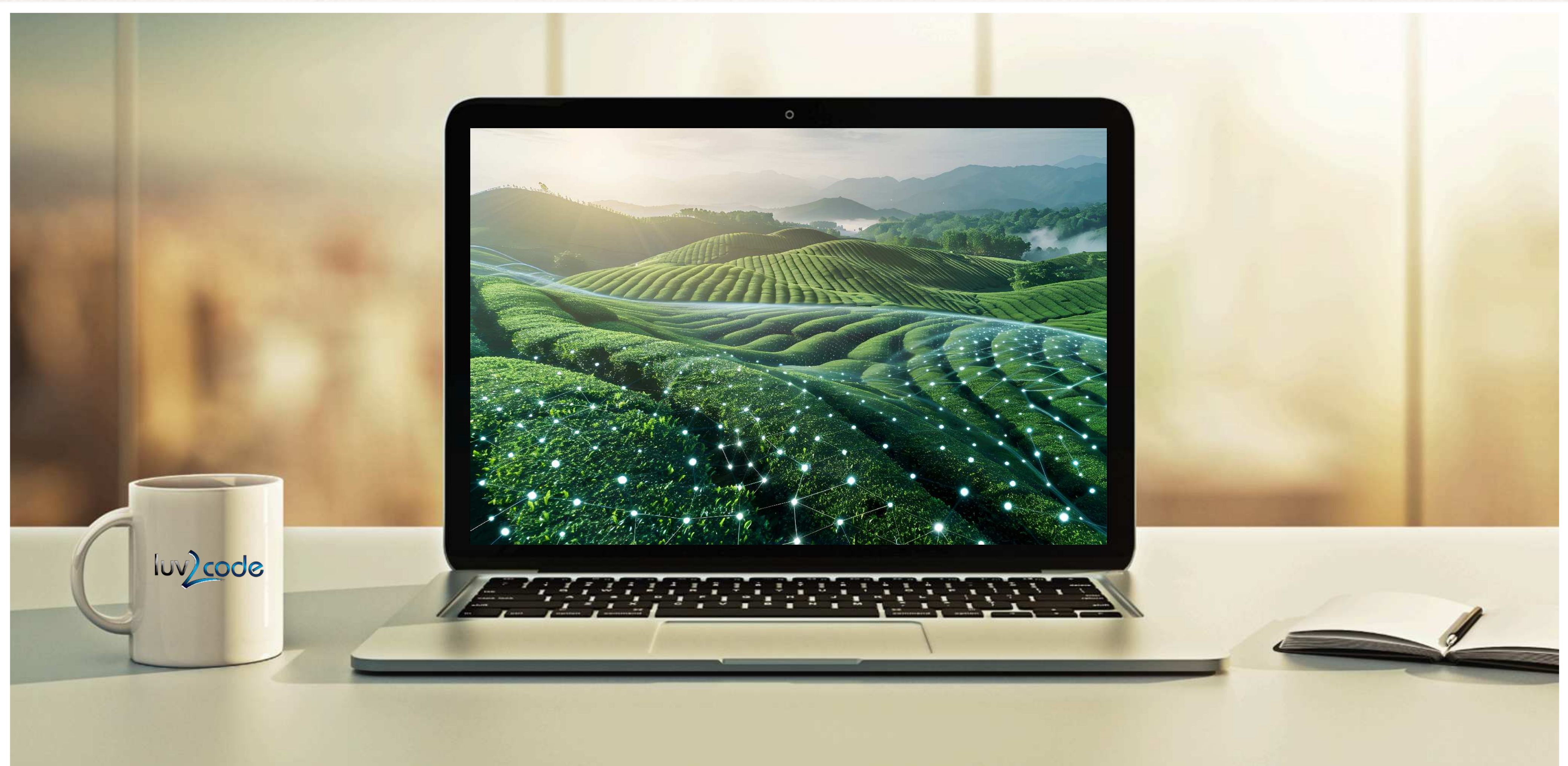
    public TodoController(TodoService todoService) {
        this.todoService = todoService;
    }

    @Operation(summary = "Delete todo for user", description = "Delete todo for the signed in user")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @DeleteMapping("/{id}")
    public void deleteTodo(@PathVariable @Min(1) long id) {
        todoService.deleteTodo(id);
    }

    <...Other Code...>

}
```


Admins



Development Process

Step-By-Step

1. Create new AdminService
2. Create AdminServiceImpl for business logic.
3. Update security config to check for Admin role for admin endpoints
4. Create AdminController to have this new endpoint

AdminService

File: AdminService.java

```
public interface AdminService {  
    List<UserResponse> getAllUsers();  
    UserResponse promoteToAdmin(long userId);  
    void deleteNonAdminUser(long id);  
}
```

- Admin users will have granted permissions to these three new functions.
- We now need to add the business logic for these by creating a new class AdminServiceImpl.

AdminServiceImpl

File: AdminServiceImpl.java

```
@Service
public class AdminServiceImpl implements AdminService {

    private final UserRepository userRepository;

    public AdminServiceImpl(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    @Transactional(readOnly = true)
    public List<UserResponse> getAllUsers() {
        return StreamSupport.stream(userRepository.findAll().spliterator(), false)
            .map(this::convertToUserResponse).toList();
    }
}
```


AdminServiceImpl

File: AdminServiceImpl.java

```
@Override
@Transactional
public UserResponse promoteToAdmin(long userId) {
    Optional<User> user = userRepository.findById(userId);

    if (user.isEmpty() || user.get().getAuthorities().stream().anyMatch(authority -> "ROLE_ADMIN"
        .equals(authority.getAuthority())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "User does not exist or already an admin");
    }

    List<Authority> authorities = new ArrayList<>();
    authorities.add(new Authority("ROLE_EMPLOYEE"));
    authorities.add(new Authority("ROLE_ADMIN"));
    user.get().setAuthorities(authorities);

    User savedUser = userRepository.save(user.get());

    return convertToUserResponse(savedUser);
}
```


AdminServiceImpl

File: AdminServiceImpl.java

```
@Override
@Transactional
public void deleteNonAdminUser(long userId) {

    Optional<User> user = userRepository.findById(userId);

    if (user.isEmpty() || user.get().getAuthorities().stream().anyMatch(authority -> "ROLE_ADMIN"
        .equals(authority.getAuthority())) {

        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "User does not exist or already an admin");
    }

    userRepository.delete(user.get());
}
```


SecurityConfig

File: SecurityConfig.java

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(configurer ->
        configurer
            .requestMatchers("/api/auth/**", "/swagger-ui/**", "/v3/api-docs/**",
                "/swagger-resources/**", "/webjars/**", "/docs").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN").anyRequest().authenticated());

    http.csrf(csrf -> csrf.disable());

    http.exceptionHandling(exceptionHandling ->
        exceptionHandling
            .authenticationEntryPoint(authenticationEntryPoint()));

    http.sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    http.addFilterBefore(jwtAuthenticationTokenFilter, UsernamePasswordAuthenticationTokenFilter.class);

    return http.build();
}
```

NEW
requestMatcher

NEW hasRole

AdminController

File: AdminController.java

```
@Tag(name = "Admin REST API Endpoints", description = "Operations related to a admin")
@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final AdminService adminService;

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    }

    @Operation(summary = "Get all users", description = "Retrieve a list of all users in the system")
    @ResponseStatus(HttpStatus.OK)
    @GetMapping
    public List<UserResponse> getAllUsers() {
        return adminService.getAllUsers();
    }
}
```


AdminController

File: AdminController.java

```
@Tag(name = "Admin REST API Endpoints", description = "Operations related to a admin")
@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final AdminService adminService;

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    }

    @Operation(summary = "Promote user to admin", description = "Promote user to admin role")
    @ResponseStatus(HttpStatus.OK)
    @PutMapping("/{userId}/role")
    public UserResponse promoteToAdmin(@PathVariable @Min(1) long userId) {
        return adminService.promoteToAdmin(userId);
    }

    <... Other Code ...>
}
```


AdminController

File: AdminController.java

```
@Tag(name = "Admin REST API Endpoints", description = "Operations related to a admin")
@RestController
@RequestMapping("/api/admin")
public class AdminController {

    private final AdminService adminService;

    public AdminController(AdminService adminService) {
        this.adminService = adminService;
    }

    @Operation(summary = "Delete user", description = "Delete a non-admin user from the system")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    @DeleteMapping("/{userId}")
    public void deleteUser(@PathVariable @Min(1) long userId) {
        adminService.deleteNonAdminUser(userId);
    }

    <... Other Code ...>
}
```